Zenzy – Specification Document

1. Problem analysis & requirements elicitation

1.1 Problem statement

Young people who want to meet in real life juggle several apps: a messaging app to coordinate, a map app to find places, and social media to discover events and venues. Location sharing in general-purpose apps is all-or-nothing and gives users little control over who sees them and when. Local shops have no lightweight channel to reach nearby people with time-limited offers.

In our survey (n = 15), 60% of respondents ask friends "where are you?" at least weekly, 73% have missed an event because they did not know about it, and 67% worry about being tracked or exposing where they live.

Zenzy is a mobile app (Flutter, Android and iOS) that combines a real-time friends map, privacy-first location sharing (Ghost Mode, close friends, time-limited sharing), group events, a community feed, and local shop discovery with flash deals in one place. The core is user-generated (consumer-to-consumer); the shop channel is a lightweight secondary feature.

1.2 Measurable goals

ID	Goal
G-01	A user sees friends' locations with one action (opening the map).
G-02	A user finds an event within 5 km in at most 3 actions.
G-03	A user turns location sharing or Ghost Mode on or off in at most 2 taps.
G-04	A shop owner publishes a flash deal in at most 5 steps.

1.3 Stakeholders

Stakeholder	Interest	Influence
End-users (mainly students and young adults)	Meet friends, join events, post, find shops, control their privacy	High
Shop owners (an End-user who creates a shop)	Promote the shop and flash deals to nearby users	Medium
Event hosts (an End-user who creates an event)	Organise events and manage participants	Medium
System Admin / moderators	Keep content safe, approve shops, manage accounts	High
Development team and course instructor	Deliver and evaluate the system	High

Stakeholder	Interest	Influence
Third-party services (map, push notification, cloud hosting, Google sign-in)	Provide the technical foundation	Low

1.4 Personas

Composite personas built from the survey groups in Section 1.5.

- **Minh, 20, university student (End-user).** Represents the 8 students, which include all 5 respondents who ask "where are you?" daily. 7 of the 8 rated their willingness to share location 4 or 5 out of 5. Main concerns: battery drain and being tracked. Prefers Close Friends (5 of 8).
- **Lan, 21, student and event organiser (End-user acting as host).** 6 of the 8 students rated their interest in creating events 5 out of 5. Needs to approve join requests and remove members who do not show up.
- **Vy, 27, office worker (End-user).** Represents the 4 office workers: all four prefer time-limited location sharing, all four worry about being tracked or exposing their home, and all four listed Flash Deal and nearby places in their top three features.
- **Hung, 31, café owner (End-user acting as shop owner).** Represents the 2 business owners: both rated map offers 5 out of 5, both listed Flash Deal and Events in their top three, both worry about exposing their location and being targeted by ads, and both prefer blocking unwanted users.
- **Thao, 26, platform admin.** Not surveyed. Reviews reported content, approves new shops, and handles abusive accounts.

1.5 Requirements elicitation

Two methods were used: an online survey (primary) and a competitor analysis.

Online survey. 15 respondents. Age: 18–22 (7), 23–28 (4), 29–35 (2), under 18 (1), over 35 (1). Occupation: students (8), office workers (4), business owners (2), other (1).

Question	Result	Requirement derived
How do you find out where friends are?	12 of 15 (80%) message them (Messenger, Zalo, or direct messages); 6 use Google Maps, 4 use Snap Map, 2 use Life360	Shared real-time friends map (UC-03)
How often do you ask "where are you?"	Daily 5, several times a week 4, several times a month 3, rarely 3 (60% at least weekly)	Map is a core feature (UC-03)
Have you missed an event because you did not know about it?	11 of 15 (73%) yes	Event discovery and notifications (UC-08, UC-22)
Interest in creating events (1–5)	Average 4.0; 10 of 15 rated 4 or 5	Any user can host an event (UC-11)
Willingness to share location (1–5)	Average 3.6; 8 of 15 rated 4 or 5; 2 of 15 rated 1 or 2	Sharing is opt-in and controllable (UC-23)

Question	Result	Requirement derived
What worries you about sharing location?	Being tracked 6, exposing home 6, battery drain 3, being targeted by ads 2, no concern 2. 10 of 15 (67%) worry about tracking or exposing home	Privacy controls (UC-07, UC-23); battery limits (Section 3.2)
Preferred privacy control	Close Friends 5, Ghost Mode 4, time-limited sharing 4, blocking users 2	Close friends and block (UC-05), Ghost Mode (UC-07), audience and time limit (UC-23)
Interest in offers on the map (1–5)	Average 4.3; 13 of 15 rated 4 or 5	Flash deals on the map (UC-13, UC-15)
Top 3 features (number of respondents who listed it)	Events 13, Flash Deal 8, Real-time Map 7, Chat 6, nearby places 6, Feed 5	MoSCoW priorities in 2.2; chat (UC-06), shops (UC-12), community feed (UC-21)

Segment patterns: all 5 daily askers are students; all 4 office workers prefer time-limited sharing; both shop owners want to create events and rated map offers 5 out of 5.

Limitations. The sample is small (n = 15) and 8 of 15 respondents are students, so the figures are indicative, not statistically representative. One respondent is under 18, which is why NFR-20 covers minors.

Competitor analysis.

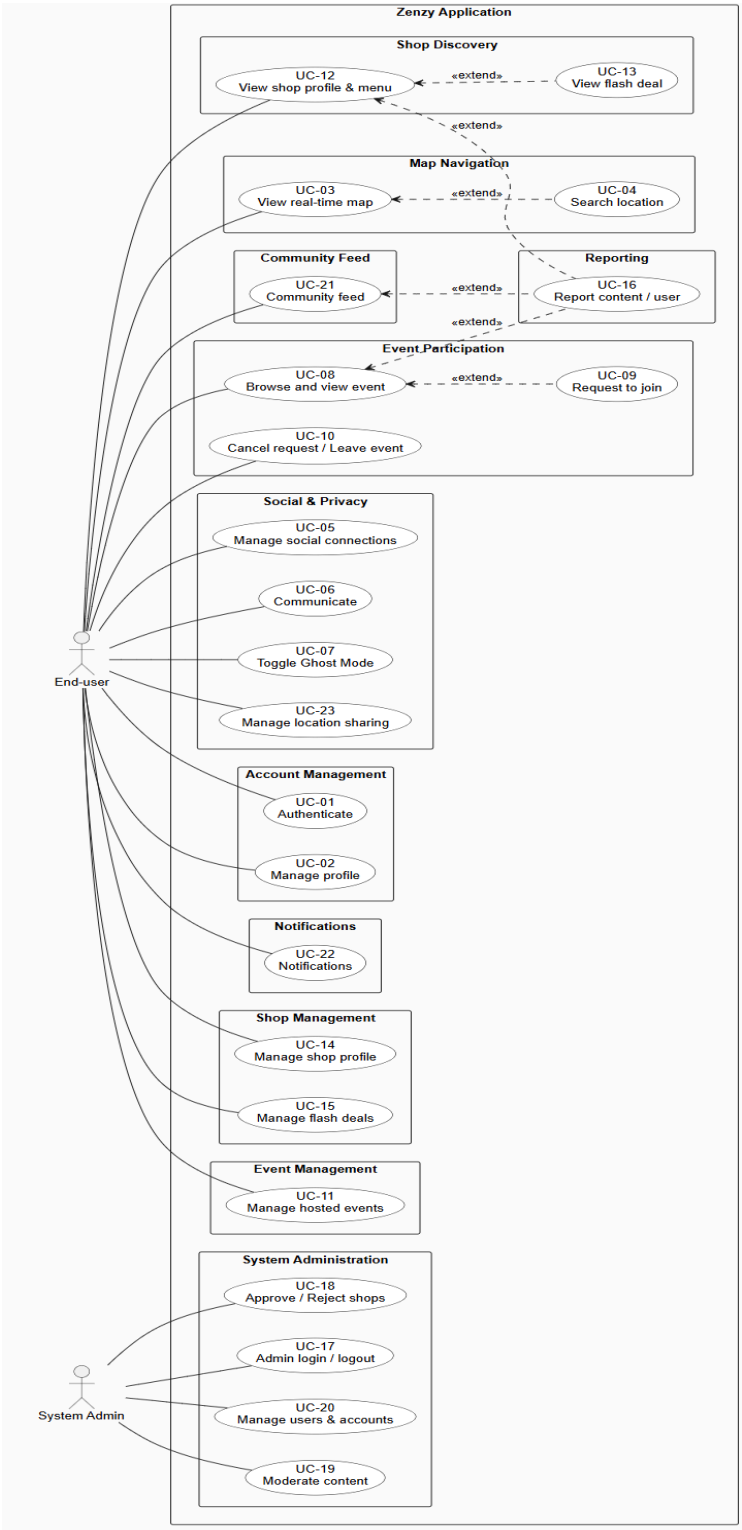
App	Strengths	Weaknesses relevant to Zenzy
Snap Map	Large user base, friends map	Tied to Snapchat; no shop deals or event management
Zenly (discontinued)	Playful friends map, ghost mode	No longer available; no shop or event features
Life360	Reliable location sharing	Aimed at family safety, not social discovery
Facebook	Strongest event discovery: 12 of 15 respondents use it to find events	No real-time map; weak privacy control for location
Google Maps	Best place data; 6 of 15 respondents use it to find friends	Limited social and event layer

Gap identified: no single app in this list combines privacy-controlled live location, a community feed, events, and local shop deals. This gap drives the feature set in Section 2.

Open item: re-check each competitor's current features before submitting.

2. Functional requirements specification

2.1 Usecase diagram



.

The system has two actors: **End-user** and **System Admin**. Shop Owner and Event Host are not separate accounts. They are contextual roles an End-user obtains by creating a shop or an event (see `shops.owner_id` and `events.host_id` in the data model in Section 5). Permissions such as "only the host can remove members" are enforced by the server and stated as preconditions in the specifications in 2.5. Before logging in, a visitor can only register or log in (UC-01), so no separate Guest actor is modelled.

Relationships: UC-04 extends UC-03; UC-09 extends UC-08; UC-13 extends UC-12; UC-16 extends UC-08, UC-12 and UC-21.

2.2 Use case list and prioritisation (MoSCoW)

Priorities follow the survey results in Section 1.5.

ID	Use case	Actor	Priority	Survey evidence
UC-01	Authenticate (register, log in, log out) with username and password, or with Google	End-user	Must	Prerequisite for all features
UC-02	Manage profile (edit, change password, view)	End-user	Must	Prerequisite for social features
UC-03	View real-time map	End-user	Must	Top-3 feature for 7 of 15; 9 of 15 ask friends' location at least weekly
UC-04	Search location (extends UC-03)	End-user	Should	Nearby places top-3 for 6 of 15
UC-05	Manage social connections (add friend, accept or reject, close friend, block)	End-user	Must	Close Friends preferred by 5 of 15; blocking by 2 of 15
UC-06	Communicate (direct and group chat)	End-user	Must	Chat top-3 for 6 of 15
UC-07	Toggle Ghost Mode	End-user	Must	Ghost Mode preferred by 4 of 15; 10 of 15 worry about tracking or exposing home
UC-08	Browse and view event	End-user	Must	Events top-3 for 13 of 15; 11 of 15 missed an event
UC-09	Request to join (extends UC-08)	End-user	Must	Same as UC-08
UC-10	Cancel request / Leave event	End-user	Should	Completes the join flow of UC-09
UC-11	Manage hosted events (create event or live pin, edit, cancel, approve requests, remove members)	End-user (host)	Must	Interest in creating events averages 4.0 of 5

ID	Use case	Actor	Priority	Survey evidence
UC-12	View shop profile and menu	End-user	Should	Nearby places top-3 for 6 of 15
UC-13	View flash deal (extends UC-12)	End-user	Should	Flash Deal top-3 for 8 of 15; map offers average 4.3 of 5
UC-14	Manage shop profile (create, edit shop and menu)	End-user (shop owner)	Should	Both business owners rated map offers 5 of 5
UC-15	Manage flash deals	End-user (shop owner)	Should	Flash Deal top-3 for both business owners
UC-16	Report content or user	End-user	Must	Needed for user-generated content (BR-03)
UC-17	Admin login / logout	System Admin	Must	Prerequisite for administration
UC-18	Approve / reject shops	System Admin	Should	Shops are pre-moderated (BR-04)
UC-19	Moderate content (review reports, hide or remove content)	System Admin	Must	Handles reports from UC-16
UC-20	Manage users and accounts (suspend, restore)	System Admin	Must	Handles abusive accounts
UC-21	Community feed (view, post, like, comment, save)	End-user	Should	Feed top-3 for 5 of 15
UC-22	Notifications (view, manage settings)	End-user	Must	11 of 15 missed an event; join and friend requests need alerts
UC-23	Manage location sharing (audience, optional time limit, location permission)	End-user	Must	Time-limited sharing preferred by 4 of 15; close friends by 5 of 15

Won't have in this release: in-app payments and ordering, shop reviews, video chat, community-contributed places, an admin mobile app, password reset by email or SMS, and sign-in with providers other than Google.

2.3 Business rules

ID	Rule
BR-01	Events, shops and posts are checked against a banned-keyword list when created or edited; a match is rejected and nothing is saved.
BR-02	A user can have at most 5 active events at the same time.

ID	Rule
BR-03	Content reported by 3 or more different users is hidden automatically until an Admin reviews it.
BR-04	Shops are pre-moderated: a new shop stays PENDING and invisible until an Admin approves it. Events and posts are post-moderated.
BR-05	Only each user's latest location is stored, with an expiry of 15 minutes; no location history is kept.
BR-06	Only the host of an event can approve requests, edit the event or remove members.
BR-07	Every Admin action is written to an insert-only audit log.

2.4 User stories with acceptance criteria

Story	Acceptance criteria (testable)
US-01 As an End-user, I want to register and log in so that I can use the app. (UC-01)	Given a unique username and a password of at least 8 characters, when I submit, then my account is created and I am logged in. Given a username already in use, then registration is rejected with a message. Given a wrong password, then login fails and no session is created; repeated failures are rate limited (NFR-08). Given I choose Google sign-in, then I am logged in without a password and, on my first sign-in, I choose a unique username.
US-03 As an End-user, I want to see my friends on a live map so that I can meet them easily. (UC-03)	Given location permission, when I open the map on 4G, then it is interactive within 3 seconds (p95) and shows my accepted friends who are not in Ghost Mode and have not blocked me. A friend's position is visible within 5 seconds of being sent (p95).
US-05 As an End-user, I want to block someone so that they cannot see or contact me. (UC-05)	Given I blocked A, then A cannot see my location, profile or events and cannot message or send me a request. Given I unblock A, then A does not automatically become a friend again.
US-06 As an End-user, I want to chat with friends and groups. (UC-06)	Given both users are online, then a message arrives within 2 seconds (p95). Given I am offline, then messages are queued and sent when the connection returns.
US-07 As an End-user, I want to turn on Ghost Mode so that nobody sees my location. (UC-07)	Given Ghost Mode is on, then no other user can retrieve my position and the app sends no location, within 5 seconds. Given it is turned off, then friends see me again. The toggle takes at most 2 taps.

Story	Acceptance criteria (testable)
US-08 As an End-user, I want to browse events near me. (UC-08)	Given a 5 km radius, when I open the map, then I see active events in that radius. Ended events are not shown. An event set to Close Friends is invisible to anyone who is not on the host's close-friends list.
US-09 As an End-user, I want to request to join an event. (UC-09)	Given an event with status OPEN, when I send a request, then my status is PENDING and the host is notified within 10 seconds. Given the event is FULL or CANCELLED, then the request button is disabled. I cannot send a second request for the same event.
US-11 As an event host, I want to approve requests and remove members. (UC-11)	Given I am the host, when I approve a request, then the member status becomes APPROVED, and when approved members reach the maximum, the event becomes FULL. Given I am not the host, then these actions are unavailable and the server returns 403. Title is 3 to 80 characters; creating a sixth active event is refused.
US-13 As an End-user, I want to see flash deals on the map so that I can use nearby offers. (UC-13)	Given a shop has an active flash deal, then its map marker shows a deal badge, and the shop profile shows the title, the discount and the remaining time. Given a deal's end time has passed, then it is no longer shown.
US-14 As a shop owner, I want to create a shop so that nearby users can find it. (UC-14)	Given valid shop information, when I submit, then the shop is created with status PENDING and is not visible to other users until an Admin approves it.
US-15 As a shop owner, I want to create a flash deal so that nearby users see my offer. (UC-15)	Given my shop is APPROVED, when I create a deal with a title, a discount and an end time later than the start time, then it becomes ACTIVE and appears on the map; when the end time passes, it becomes ENDED automatically. Given my shop is PENDING, then I cannot create a deal.
US-16 As an End-user, I want to report abusive content. (UC-16)	Given I report an item, then I choose a reason and the report reaches the Admin queue within 5 seconds. I can report an item only once and cannot report my own. Given 3 different users report it, then it is hidden.
US-18 As an Admin, I want to approve or reject shops so that only legitimate shops appear. (UC-18)	Given a PENDING shop, when I approve it, then it becomes visible on the map. When I reject it, then the owner is notified with a reason.
US-19 As an Admin, I want to review reports and remove violating content. (UC-19, UC-20)	Given a report, then I can dismiss it, remove the content, or remove it and suspend the account, with a

Story	Acceptance criteria (testable)
	reason. The action is logged, the content disappears within 10 seconds, and the owner is notified with the reason. A normal user calling admin functions gets 403.
US-21 As an End-user, I want to read and share posts so that I can follow my community. (UC-21)	Given I open the feed, then I see the 20 newest posts I am allowed to see, newest first, and scrolling loads the next 20. Given text of up to 500 characters and up to 4 photos, when I post, then it appears in the feed within 5 seconds, unless it contains a banned keyword (BR-01), in which case nothing is saved. I can like, comment on and save a post. Hidden posts (BR-03) and posts of users who blocked me never appear.
US-22 As an End-user, I want notifications so that I do not miss requests and invitations. (UC-22)	Given a join request, friend request or moderation result, then a push notification arrives within 10 seconds (p95) and an entry appears in the list. Given I turn a type off, then I receive no push for it.
US-23 As an End-user, I want to choose who sees my location and for how long. (UC-23)	Given I select friends or close friends and optionally a duration, then only that audience sees me, and sharing stops automatically when the duration ends. Location sharing is off until I agree.

2.5 Use case specifications

UC-01: Authenticate

Field	Description
Actor	End-user (before login: an unregistered visitor)
Precondition	App installed, network available
Main flow	1. User chooses Register, Log in, or Continue with Google. 2. For Register, user enters a username, a password (at least 8 characters), and a date of birth. For Log in, user enters username and password. For Google, user selects a Google account. 3. System validates the input and creates the account or the session. 4. On a first Google sign-in, the user also chooses a unique username and enters a date of birth. 5. System opens the map screen. To log out, user taps Log out and the system ends the session.
Alternative flows	Username already used: system rejects with a message. Wrong password: error, no session. Repeated failures: attempts are rate limited (NFR-08). Google sign-in cancelled: user returns to the login screen.

Field	Description
	Network unavailable: error and a retry option. User under 16: see NFR-20.
Postcondition	User has an active session (none after log out)

UC-02: Manage profile

Field	Description
Actor	End-user
Precondition	User is logged in
Main flow	1. User opens the profile screen. 2. User views the profile or taps Edit. 3. User changes display name, avatar, or bio, or taps Change password and enters the current and the new password. 4. System validates and saves.
Alternative flows	Current password wrong: password is not changed. Account created with Google has no password: Change password is hidden. Image over 500 KB: the app compresses it before upload.
Postcondition	Profile data is updated

UC-03: View real-time map

Field	Description
Actor	End-user
Precondition	The End-user is logged in and has granted location permission
Main flow	1. User opens the Map tab. 2. System gets the current position. 3. System loads visible friends, events and shops within 5 km. 4. System draws markers and keeps friend positions updated in real time. 5. User taps a marker to see a summary.
Alternative flows	A1. No friends yet: empty state with a hint to add friends. A2. User filters by Friends, Events or Shops. A3. User searches a location (UC-04).
Exceptions	E1. Permission denied: the map still opens without the user's marker and shows how to enable permission. E2. GPS off: prompt to turn it on. E3. No network: cached data with an offline banner.
Postcondition	The map shows only data the user is allowed to see

UC-04: Search location (extends UC-03)

Field	Description
Actor	End-user
Precondition	Map is open
Main flow	1. User taps the search box and enters a keyword. 2. System lists matching places (shops and addresses). 3. User selects a result. 4. Map centres on the selected place.
Alternative flows	No results: system shows a "no results" message.
Postcondition	Map is centred on the selected place

UC-05: Manage social connections

Field	Description
Actor	End-user
Precondition	User is logged in
Main flow	1. User searches another user by username. 2. User sends a friend request. 3. Recipient accepts or rejects it. 4. User may mark an accepted friend as a close friend, or block any user from their profile.
Alternative flows	Request already pending: the duplicate is rejected. Blocking: any friendship ends, the blocked user disappears from the blocker's map, search, chat and feed, and cannot send requests or messages. Unblocking does not restore the friendship.
Postcondition	Relationship status (PENDING, ACCEPTED, BLOCKED) and the close-friend flag are saved

UC-06: Communicate

Field	Description
Actor	End-user
Precondition	User is logged in. Direct chat needs an accepted friend who has not blocked the user; group chat needs an approved member of the event.
Main flow	1. User opens a chat (a friend or an event group). 2. User types a message and sends it. 3. System stores the message and delivers it to online recipients. 4. Recipients who are not in the app receive a push notification (UC-22).
Alternative flows	Offline: the message is queued and sent when the connection returns (NFR-06). Recipient has blocked the sender: the message is not delivered.
Postcondition	The message is stored in the chat history

UC-07: Toggle Ghost Mode

Field	Description
Actor	End-user
Precondition	The End-user is logged in
Trigger	The End-user switches the Ghost Mode toggle
Main flow	1. User opens the privacy settings or the map. 2. User switches Ghost Mode on. 3. The app stops location updates and shows a Ghost Mode indicator at once. 4. System saves the setting and deletes the user's latest location. 5. Friends' maps remove the user's marker.
Alternative flows	Network unavailable at step 4: the toggle stays applied locally, the setting is marked pending and retried when the connection returns; no location is sent meanwhile.
Postcondition	The user's location is hidden from all other users

UC-08: Browse and view event

Field	Description
Actor	End-user
Precondition	User is logged in
Main flow	1. User opens the Map tab or the Events list. 2. System returns events within 5 km with status OPEN or FULL that the user may see. Events set to Close Friends are visible only to the host's close friends. 3. User opens an event to see title, type, time, place, description, host, and number of participants.
Alternative flows	No events nearby: empty state suggesting to create one. User filters events by type.
Postcondition	Event details are displayed

UC-09: Request to join (extends UC-08)

Field	Description
Actor	End-user (becomes a member if approved)
Precondition	Logged in; viewing an event that has not ended
Main flow	1. User taps Join. 2. System checks that the event is OPEN, not full, that there is no earlier request, and that the user is not blocked by the host. 3. System creates a PENDING request. 4. Host is notified. 5. The button changes to Cancel request.

Field	Description
Alternative flows	Host approves: user becomes a member, is added to the event chat and is notified. Host rejects: user is notified.
Exceptions	Event full, duplicate request, blocked by host, or no network: no request is created and a clear message is shown.
Postcondition	The request exists once, with status PENDING, APPROVED or REJECTED

UC-10: Cancel request / Leave event

Field	Description
Actor	End-user
Precondition	User has a PENDING or APPROVED participation
Main flow	1. User opens the event. 2. User taps Cancel request (if PENDING) or Leave event (if APPROVED). 3. System sets the status to CANCELLED or LEFT. 4. If the event was FULL, it becomes OPEN again. 5. System notifies the host.
Alternative flows	Event already started: leaving is still allowed and the host is notified.
Postcondition	Participation status is updated and the seat is released

UC-11: Manage hosted events

Field	Description
Actor	End-user (acting as the event's host)
Precondition	Logged in. For edit, approve, remove and cancel actions, the user is the host (`events.host_id` equals the user's ID).
Main flow	1. User creates an event or live pin with title, type, time, place, description, audience and optional maximum participants. 2. System applies BR-01 and BR-02, then saves it with status OPEN. 3. Other users send join requests. 4. Host approves or rejects each request. 5. Host may edit the event, remove a member, or cancel the event.
Alternative flows	Approved members reach the maximum: the event becomes FULL. Host removes a member: participation becomes KICKED and the seat is released. Host cancels: the event becomes CANCELLED and members are notified.
Exceptions	Banned keyword, missing fields, sixth active event, or no network: nothing is saved and the entered data is kept for retry.
Postcondition	Event and participant statuses are updated

UC-12: View shop profile and menu

Field	Description
Actor	End-user
Precondition	User is logged in
Main flow	1. User taps a shop marker on the map or selects a shop from search. 2. System shows the shop name, description, address, and menu items with prices.
Alternative flows	Shop is not APPROVED: it is not visible, except to its owner (BR-04).
Postcondition	Shop information is displayed

UC-13: View flash deal (extends UC-12)

Field	Description
Actor	End-user
Precondition	The shop has an ACTIVE flash deal
Main flow	1. The map shows a deal badge on shops with an active deal. 2. On the shop profile, user taps the deal banner. 3. System shows the title, the discount, and the remaining time.
Alternative flows	Deal has ended: the badge and the banner disappear.
Postcondition	Deal details are displayed

UC-14: Manage shop profile

Field	Description
Actor	End-user (acting as shop owner)
Precondition	User is logged in. To edit, the user is the owner of that shop.
Main flow	1. User chooses Create shop. 2. User enters name, description, address, location on the map, and menu items (name, price, optional photo). 3. System applies BR-01, saves the shop with status PENDING (BR-04), and adds it to the Admin queue. 4. The owner can edit the shop information and menu at any time.
Alternative flows	Required field missing or banned keyword: nothing is saved and the system shows an error. Shop REJECTED: owner sees the reason, edits, and resubmits.
Postcondition	Shop exists and becomes visible to others only after APPROVED

UC-15: Manage flash deals

Field	Description
Actor	End-user (acting as shop owner)
Precondition	User owns an APPROVED shop
Main flow	1. Owner chooses Create flash deal. 2. Owner enters title, discount, start time, and end time. 3. System saves it as ACTIVE and shows it on the map. 4. Owner may edit the deal or end it early.
Alternative flows	End time not later than start time: system shows an error. Shop not APPROVED: the option is unavailable. Owner ends the deal early: status becomes CANCELLED.
Postcondition	Deal is ACTIVE until its end time, then ENDED automatically

UC-16: Report content or user

Field	Description
Actor	End-user
Precondition	Logged in; viewing content or a profile that is not the user's own
Main flow	1. User chooses Report. 2. User selects a reason (spam, scam, harassment, illegal content, sensitive content, other) and an optional note. 3. System stores the report. 4. System confirms.
Alternative flows	Already reported by this user: the system acknowledges without creating a duplicate. Third distinct report: the content is hidden pending review (BR-03).
Postcondition	The report is in the Admin queue

UC-17: Admin login / logout

Field	Description
Actor	System Admin
Precondition	An account with role admin exists
Main flow	1. Admin opens the admin dashboard and enters credentials. 2. System verifies them and checks that the role is admin. 3. System opens the dashboard. To log out, Admin taps Log out and the system ends the session.
Alternative flows	Credentials of a non-admin account: access is denied.
Postcondition	Admin session is active (none after log out)

UC-18: Approve / reject shops

Field	Description
Actor	System Admin
Precondition	Admin is logged in; at least one shop has status PENDING
Main flow	1. Admin opens the pending shop list. 2. Admin reviews the shop information. 3. Admin approves. 4. System sets APPROVED, records `reviewed_by`, writes the audit log (BR-07), and notifies the owner.
Alternative flows	Admin rejects and enters a reason: system sets REJECTED and notifies the owner.
Postcondition	Only APPROVED shops are visible to other users

UC-19: Moderate content

Field	Description
Actor	System Admin
Precondition	Admin is logged in to the dashboard
Main flow	1. Admin opens the report queue, sorted by number of reports and time. 2. Admin reviews the content and context. 3. Admin chooses: dismiss, remove, remove and warn, or remove and suspend the account (UC-20). 4. Admin enters a reason. 5. System applies the action, writes the audit log, and notifies the content owner.
Exceptions	Content already deleted by its owner: the report is marked resolved. Two Admins act at once: only the first action takes effect.
Postcondition	The report is RESOLVED and the action is in the audit log

UC-20: Manage users and accounts

Field	Description
Actor	System Admin
Precondition	Admin is logged in to the dashboard
Main flow	1. Admin searches a user by username. 2. Admin views the account status and the reports against the user. 3. Admin suspends or restores the account and enters a reason. 4. System applies the change, writes the audit log (BR-07), and notifies the user.
Exceptions	A suspended user tries to log in: access is denied and the reason is shown.
Postcondition	The account status is updated

UC-21: Community feed

Field	Description
Actor	End-user
Precondition	User is logged in
Main flow	1. User opens the Feed tab. 2. System shows the 20 newest posts the user may see, newest first. 3. User scrolls and the system loads the next 20. 4. User opens a post to read it and may like, comment on, or save it. 5. To post, user taps Create post, enters text (up to 500 characters) and optionally up to 4 photos, and taps Post; the system applies BR-01 and saves the post.
Alternative flows	No posts: empty state. Offline: cached posts with an offline banner. The author can edit or delete their own post.
Exceptions	Banned keyword: nothing is saved. Upload fails: the draft is kept for retry.
Postcondition	The post exists as ACTIVE, and may later become HIDDEN or REMOVED through moderation (BR-03, UC-19)

UC-22: Notifications

Field	Description
Actor	End-user
Precondition	User is logged in
Main flow	1. An event occurs: a join request, the result of a request, a friend request, or a moderation result. 2. System stores a notification and sends a push notification. 3. User opens the Notifications tab to see the list and taps an entry to open the related screen. 4. In the settings, user turns each notification type on or off.
Alternative flows	Notification permission denied: no push, the in-app list still updates. Type turned off: no push for it.
Postcondition	The notification is stored and marked read once opened

UC-23: Manage location sharing

Field	Description
Actor	End-user
Precondition	User is logged in
Main flow	1. User opens the sharing settings, or taps Share on the map for the first time. 2. System explains why location is needed and asks for permission. 3. User chooses the audience: friends or close friends. 4.

Field	Description
	User optionally sets a time limit: 1 hour, 8 hours, 24 hours, or until turned off. 5. System saves the settings and starts sharing. 6. When the time limit ends, sharing stops automatically.
Alternative flows	Permission denied: sharing stays off and the map works without the user's marker. Ghost Mode on: Ghost Mode takes precedence (UC-07).
Postcondition	Location is shared only with the chosen audience and only while allowed

The use case specifications above follow one template; Section 5 links them to the data model.

3. Non-functional requirements & mobile constraints

The figures below are design targets for a prototype used by about 50 people. Each one is verified by the test in the last column; items marked [verify] depend on the chosen hosting plan and MongoDB Atlas tier or on a source still to be checked.

3.1 Measurable non-functional requirements

ID	Category	Requirement	How to measure
NFR-01	Performance	The map screen is interactive within 3 s on a 4G connection (p95).	Timed test on a mid-range device
NFR-02	Performance	API responses are under 500 ms (p95) for non-media requests under normal load; the API server and the database cluster are placed in the region closest to users.	Load test report
NFR-03	Performance	Chat messages are delivered within 2 s (p95) when both users are online.	End-to-end timing test
NFR-04	Real-time	A friend's location update is visible within 5 s (p95) of being sent.	Two-device test
NFR-05	Offline	Without network, the app shows the last known friends, events, lists and recent feed posts with an offline banner within 2 s. Base-map caching is used only where the map provider's terms allow [verify].	Airplane-mode test
NFR-06	Offline	Chat messages written offline are queued and sent when the connection returns. Write actions keep entered data, allow retry and never create duplicates (idempotency key).	Airplane-mode test
NFR-07	Security	All traffic uses HTTPS (TLS 1.2 or higher). Passwords are stored only as salted bcrypt or Argon2	Security review

ID	Category	Requirement	How to measure
		hashes in a separate credentials collection and are never returned by any API. Google sign-in uses OAuth and stores no password.	
NFR-08	Security	Access tokens expire after 15 minutes and refresh tokens are rotated and expire after 30 days (implemented in the API). Failed logins are rate limited to 5 attempts per 15 minutes per account and per IP address.	Configuration check, automated test
NFR-09	Privacy	Ghost Mode and sharing audience are enforced on the server by a central authorization layer that filters every location read and every real-time event: a hidden user's location is never returned. Ghost Mode takes effect within 5 s.	API test with unauthorised accounts
NFR-10	Privacy	On account deletion, personal data and stored media are permanently removed within **72 hours**, in strict compliance with **Article 16, Clause 3 of Vietnam's Decree 13/2023/ND-CP** on Personal Data Protection (which mandates data controllers to delete personal data within 72 hours upon receiving a valid request from the data subject).	Process check
NFR-11	Availability	Backend availability is at least 95% per month, subject to the chosen hosting and Atlas plans [verify].	Uptime monitor
NFR-12	Scalability	The system supports 50 concurrent users without breaching NFR-01 to NFR-04 and is tested with 100 users for headroom, within the limits of the chosen hosting plan and Atlas tier [verify WebSocket connection capacity].	Load test
NFR-13	Accessibility	Text contrast follows WCAG AA (4.5:1), touch targets are at least 48 dp, and screen-reader labels exist on main screens.	Accessibility scanner
NFR-14	Privacy	Only the latest location per user is stored and it expires after 15 minutes; no location history exists.	Schema review, data check
NFR-15	Security	Every API route sits behind authentication middleware with ownership and role checks. The MongoDB connection string exists only on the	Code review, API test

ID	Category	Requirement	How to measure
		server and no client connects to the database. Unauthorised calls return 403.	
NFR-16	Security	The Admin audit log is insert-only and kept for at least 12 months.	Schema review
NFR-17	Usability	Friends' locations need one action; Ghost Mode needs at most 2 taps; every list and map screen has loading, empty and error states.	Action count, design review
NFR-18	Maintainability	MVVM is followed; ViewModel and Repository unit-test coverage is at least 70%; lint and tests run on every push.	Coverage report, CI
NFR-19	Localisation	English and Vietnamese UI; times are stored in UTC and shown in the device time zone.	Language switch test
NFR-20	Privacy	Registration collects the date of birth. Accounts under 16 need parental consent before location sharing is enabled. Open item: verify the exact legal requirement (Decree 13/2023/ND-CP on personal data protection) before submitting.	Registration flow test
NFR-21	Performance	The first 20 feed posts load within 2 s on 4G (p95); photos load progressively.	Timed test

3.2 Mobile constraints

Platform and OS versions. Flutter (Dart), one codebase. Android 8.0 (API 26) and above; iOS 15 and above (assumed targets; confirm against Flutter and the map SDK). Portrait layout, 360 to 430 dp wide. Reference devices: one mid-range Android phone (4 GB RAM) and one recent iPhone.

Permissions.

- Location is requested in context, with a short explanation before the system dialog. While-using-the-app is the default; background location is optional and needs a separate opt-in. Android 10 and above needs the background location permission and a visible notification while tracking; iOS needs "Always" authorization and the location background mode.
- If location is denied, the app still works for events, shops, feed and chat, and the map shows a prompt to enable location.
- Notification permission is requested after the first event request or friend request, not at first launch. Android 13 and above asks at runtime.
- Camera or photo access is requested only when the user changes an avatar or adds a picture.

Battery.

- Location is sent on an adaptive interval: every 15 s while moving and every 2 min when stationary, only while sharing is on; it stops completely in Ghost Mode.

- Target: background location adds no more than 5% battery drain per hour. This is a hypothesis to measure on the reference devices, and the interval will be raised if it fails.
- Background tracking can be turned off in the settings (3 of 15 survey respondents worry about battery drain). The operating system may pause background updates, so the map shows a "last updated" time for each friend.

Network.

- The app remains usable on 3G: images are compressed (at most 500 KB each) and lazy-loaded, and lists are paginated.
- Failed requests are retried with exponential backoff and show a clear error with a retry option.
- Recent data is cached locally to reduce mobile data use.

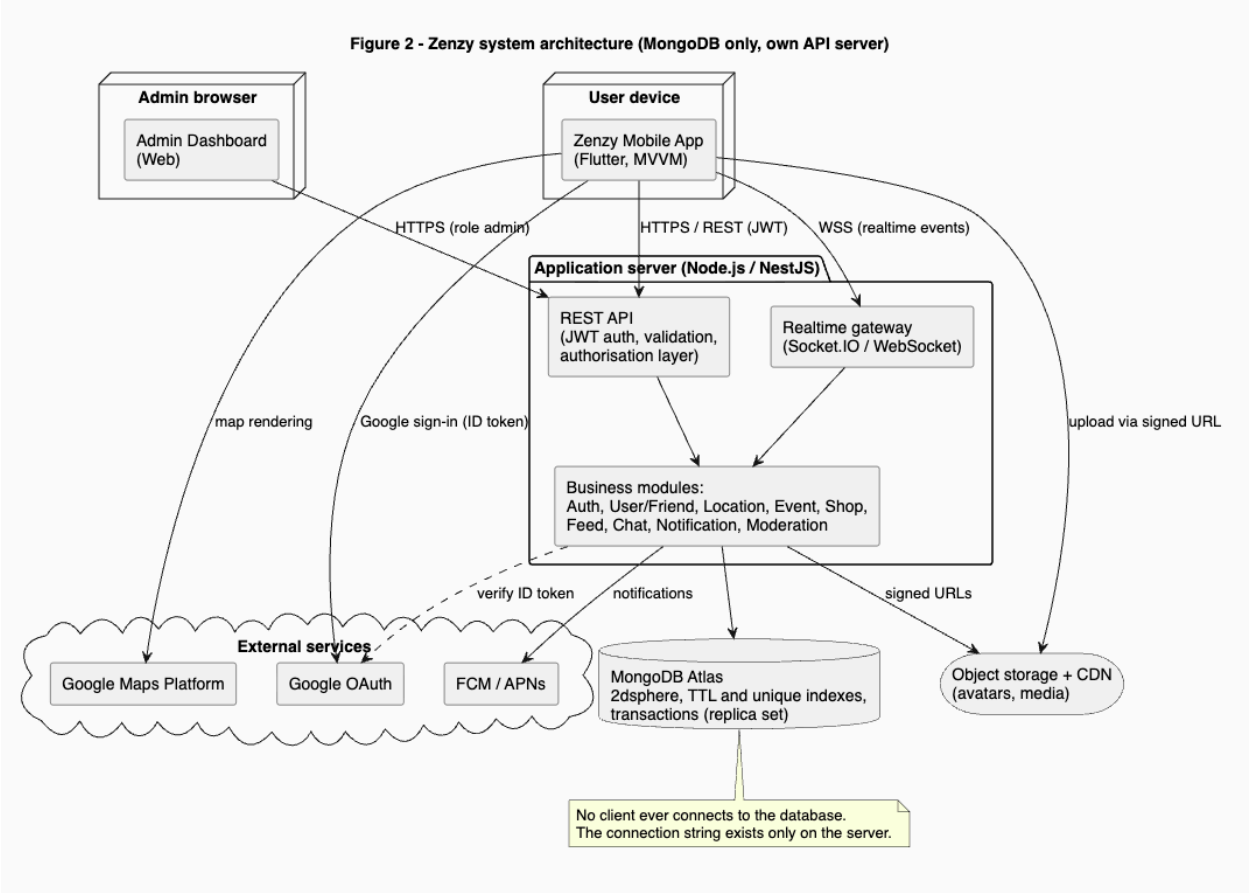
3.3 Behaviour by permission and connection state

Situation	Required behaviour
Location not yet requested	Explanation screen, then the system dialog
Permission denied	Map opens without the user's marker; guidance to open Settings
GPS off	Prompt to turn on GPS
Ghost Mode on	Clear indicator; no location sent; map and friends still visible
No network	Last data, offline banner and retry button
Loading	Skeleton screens on map and feed
No friends, events or notifications	Empty state with a suggested action
Notifications denied	App works normally; the in-app list still updates

4. System architecture design

The backend is **MongoDB only**: one database (MongoDB Atlas) behind our own API server. There is no second database and no Backend-as-a-Service. MongoDB has no row-level permissions and a client must never hold database credentials, so **all data access goes through the API server**, which also enforces privacy and permissions. The diagrams are in the file zenzy_architecture_mongodb.puml;

4.1 Overall architecture (app, backend, external services)



Component	Responsibility	Technology	Related requirements
Mobile app	UI, screen state, local cache, device location, push handling	Flutter (Dart), MVVM	All FRs, NFR-01, NFR-05, NFR-17
Admin dashboard	Report queue, shop approval, account management	Web app calling the same API with the admin role	UC-17 to UC-20
API server	Authentication (JWT), validation, business rules, a central authorisation layer, rate limiting	Node.js with NestJS (TypeScript), REST	NFR-07 to NFR-09, NFR-15
Realtime gateway	Pushes location, chat, event and notification events to allowed users	Socket.IO (WebSocket) in the API server process	NFR-03, NFR-04
MongoDB Atlas	All business data; geospatial queries; expiry of locations and tokens; uniqueness rules; transactions	MongoDB (replica set), 2dsphere, TTL and unique indexes	NFR-14, NFR-16
Object storage + CDN	Avatars, post photos, shop and menu images (only URLs are stored in MongoDB)	S3-compatible storage or Cloudinary, signed uploads	UC-02, UC-14, UC-21
Google Maps Platform	Map rendering (and place search for UC-04)	Google Maps SDK	UC-03, UC-04

Component	Responsibility	Technology	Related requirements
Push	Notifications to Android and iOS	FCM (iOS through APNs)	UC-22, NFR-04
Google OAuth	Google sign-in: the app gets an ID token and the server verifies it	OAuth	UC-01

Principle: privacy and permissions are always enforced on the server. The client hides buttons for a friendly UI but is never the protection layer.

4.2 Location handling

- The app sends the user's position (PUT /me/location) every 15 s while moving and every 2 min when stationary, only while sharing is on (see 3.2).
- user_locations holds one document per user (the _id is the user ID) with a GeoJSON point and an updated_at time. Each update overwrites the previous value, so no history exists (BR-05, NFR-14). A TTL index on updated_at removes documents after 15 minutes; MongoDB runs this cleanup about once a minute, so removal can lag by a minute or two. The API also ignores locations older than 15 minutes when reading.
- When user A opens the map, the server first finds friends who are accepted, not blocked in either direction, not in Ghost Mode, and whose sharing audience (and time limit) includes A; it then reads their documents from user_locations and returns each position with its updated_at, so the app can show the last updated time. Events and shops come from a radius query ($geoWithin with $centerSphere, or $geoNear) on a 2dsphere index. GeoJSON order is longitude then latitude.
- Real-time updates: after saving a position, the server finds the friends allowed to see it and emits an event only to their Socket.IO rooms.
- Ghost Mode: the app stops sending, the server deletes the user's location document and emits a hide event, so friends' maps remove the marker (NFR-09). When a sharing time limit ends (UC-23), the server does the same.
- If the socket drops, the app falls back to refreshing over REST at the same interval.

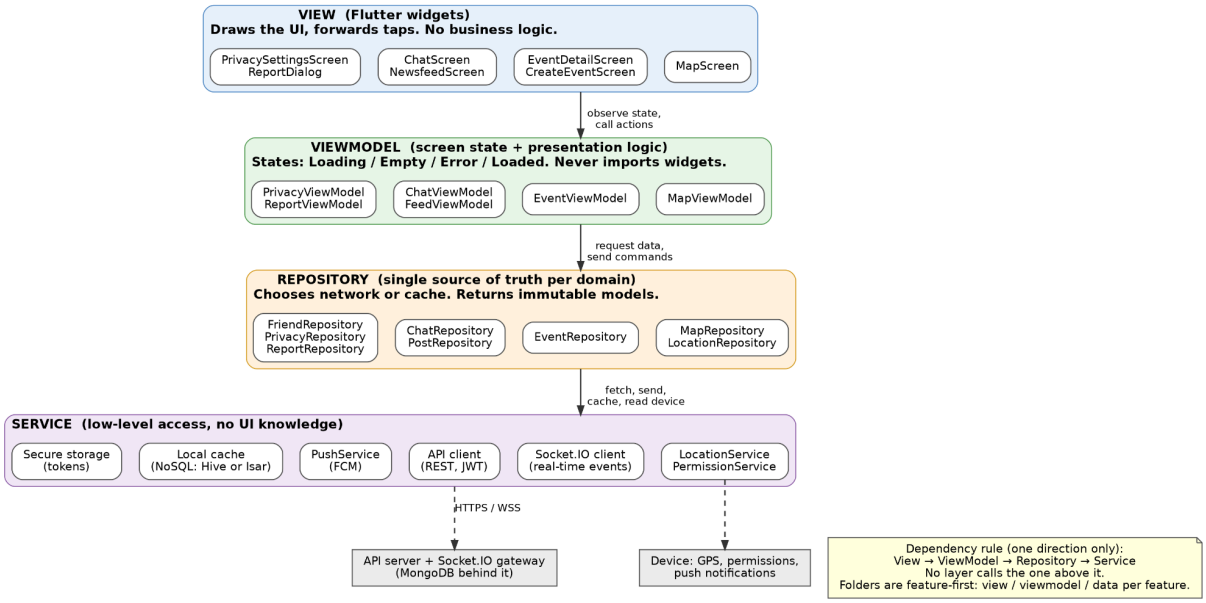
4.4 Technology choices and justification

Decision	Chosen	Alternatives considered	Why	Trade-off
Mobile framework	Flutter	Native, React Native	One codebase for iOS and Android; smooth rendering for the map screens; already used in the mockups	Background location needs some platform-specific code
In-app pattern	MVVM + Repository + Service	MVC, BLoC, full Clean Architecture	Separates View, logic and data; ViewModels are easy to unit-test (NFR-18); light enough for the team	Needs discipline on one-way dependencies
Database	MongoDB Atlas (only database)	PostgreSQL with PostGIS (for example through Supabase), Firestore	Team decision. One store means no cross-database joins. A 2dsphere index handles the 5 km query; TTL indexes implement the 15-minute location expiry (NFR-14) directly; flexible documents suit posts, events and chat; unique indexes enforce one report per user, one request per event and one friendship per pair	No row-level security and no foreign keys: authorisation and references live in server code and must be tested carefully

Decision	Chosen	Alternatives considered	Why	Trade-off
Backend	Own API server, Node.js with NestJS	Backend-as-a-Service, Express only	Required, because clients cannot connect to MongoDB. One place for authorisation, business rules and the audit log; TypeScript shares types with the API contract in Section 5	The team builds and runs authentication, real-time delivery and file handling itself
Authentication	Username and password (bcrypt or Argon2) plus Google ID token verification; JWT access and refresh tokens	Session server, managed auth service, email or phone sign-in	Matches UC-01; fits mobile and meets NFR-07 and NFR-08; refresh tokens are stored hashed and rotated	Security-critical code that must be reviewed and tested. No password reset (out of scope)
Real-time	Socket.IO on the API server	Polling, MongoDB change streams to clients	Direct control over who receives each event, which privacy filtering needs; REST polling is the fallback. One instance is enough for about 50 concurrent users (NFR-12)	Scaling beyond one instance needs a shared adapter [verify]
Media storage	S3-compatible storage or Cloudinary	GridFS inside MongoDB	CDN delivery and signed uploads; keeps large files out of the database	One more external service
Maps	Google Maps Platform	OpenStreetMap with flutter_map	Strong data for Vietnam; stable Flutter SDK	Quota and cost; API key must be restricted per app
Push	FCM	Own push service	One API for both platforms	Delivery time is not guaranteed; measure against NFR-04
Sign-in providers	Username and Google only	Facebook, phone OTP	Facebook is out of scope; phone OTP would add SMS cost	App Store rules about third-party sign-in should be checked before release

4.5 In-app architecture (MVVM)

Figure 3 – In-app architecture of Zenzy (MVVM + Repository + Service)



Layer	Responsibility	Rules	Examples in Zenzy
View	Draws UI and forwards taps	No business logic; reads state from the ViewModel	MapScreen, EventDetailScreen, CreateEventScreen
ViewModel	Holds screen state (loading, empty, error, loaded) and presentation logic	Never imports widgets; never calls the network directly	MapViewModel, EventViewModel, PrivacyViewModel
Repository	Single source of truth for one domain; combines remote data and cache	Decides network or cache; returns immutable models	MapRepository, EventRepository, FriendRepository
Service	Low-level access to the API, sockets and the device	Knows nothing about the UI	API client, Socket.IO client, LocationService, PermissionService, PushService, secure storage, local cache

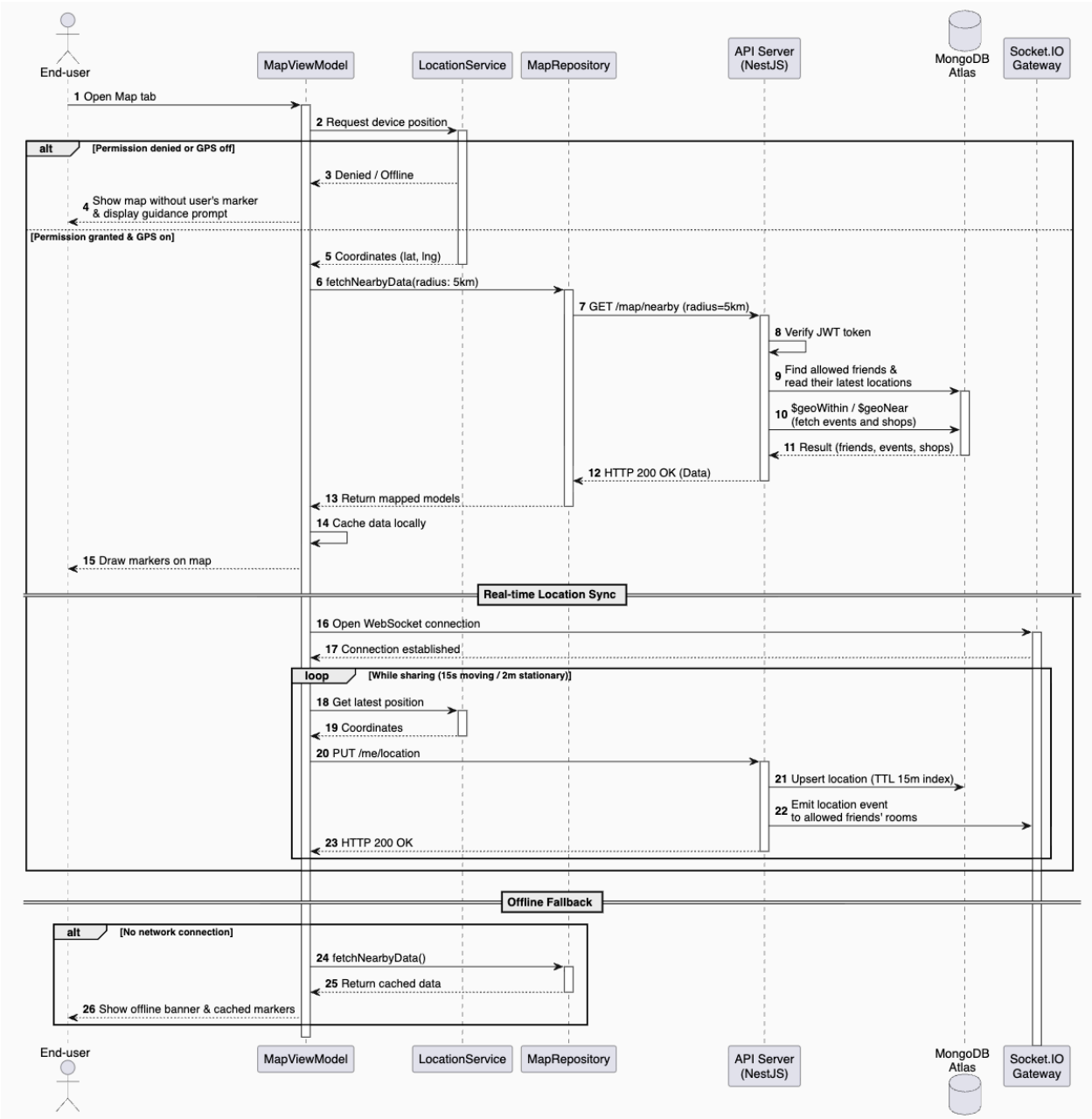
Dependencies run one way only: **View, ViewModel, Repository, Service.** Folders are organised feature-first; each feature has view, viewmodel and data folders.

COMP	Feature module	Main views	ViewModel	Repository	Backend module and collections	Use cases
COMP-01	Auth	Splash, Login, Register	AuthViewModel	AuthRepository	Auth module; users, credentials, refresh_tokens	UC-01, UC-17
COMP-02	Profile and social	Profile, Friend list, Add friend	ProfileViewModel, FriendViewModel	UserRepository, FriendRepository	User/Friend module; users, friendships, blocks	UC-02, UC-05

COMP	Feature module	Main views	ViewModel	Repository	Backend module and collections	Use cases
COMP-03	Privacy and location sharing	Privacy settings, Close friends, Blocklist	PrivacyViewModel	PrivacyRepository	User and Location modules; users, friendships	UC-07, UC-23
COMP-04	Map	Home map, Search	MapViewModel	MapRepository, LocationRepository	Location, Event and Shop modules; user_locations, events, shops	UC-03, UC-04
COMP-05	Events	Event detail, Create event, Manage requests	EventViewModel	EventRepository	Event module; events, event_participants	UC-08 to UC-11
COMP-06	Shops	Shop detail, Create shop, Flash deal	ShopViewModel	ShopRepository	Shop module; shops, flash_deals	UC-12 to UC-15
COMP-07	Feed	Newsfeed, Post detail	FeedViewModel	PostRepository	Feed module; posts, post_likes, post_comments	UC-21
COMP-08	Chat	Messages, Chat room	ChatViewModel	ChatRepository	Chat module and gateway; conversations, messages	UC-06
COMP-09	Notifications	Notifications, Settings	NotificationViewModel	NotificationRepository	Notification module; notifications, device_tokens	UC-22
COMP-10	Report	Report dialog	ReportViewModel	ReportRepository	Moderation module; reports	UC-16
COMP-11	Admin dashboard	Report queue, Shop approval, Accounts	AdminViewModel	AdminRepository	Moderation module; reports, admin_audit_log	UC-17 to UC-20

4.6 Sequence diagrams for the main flows

The five flows touch the most important requirements: the map (performance and privacy), Ghost Mode (privacy), event creation (rules and moderation), joining (host-only permission), and reporting with moderation. Endpoints are examples; the full contract is in Section 5.



SEQ-01: View real-time map (UC-03).

1. The user opens the Map tab; MapViewModel asks LocationService for the position.

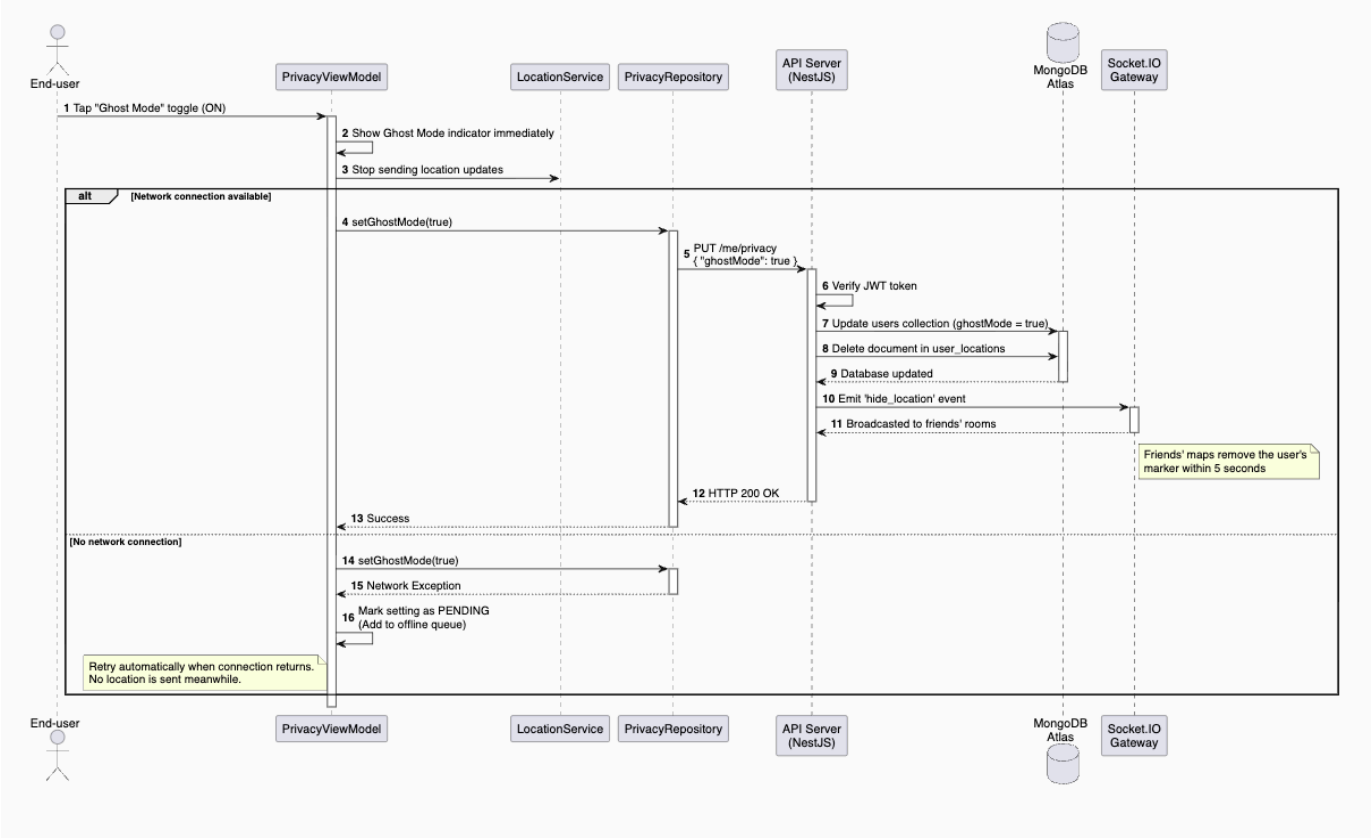
2. If permission is denied or GPS is off, the map opens without the user's marker and shows guidance.

3. Otherwise MapRepository calls GET /map/nearby with a 5 km radius.

4. The server verifies the token, finds the friends the caller may see, reads their latest locations, runs the radius query for events and shops, and returns the result; the app caches it and draws markers.

5. The app opens the socket. While sharing, it sends its own position every 15 s moving or 2 min stationary; the server forwards it only to friends who are allowed to see it.

6. Without network, cached data and an offline banner are shown.



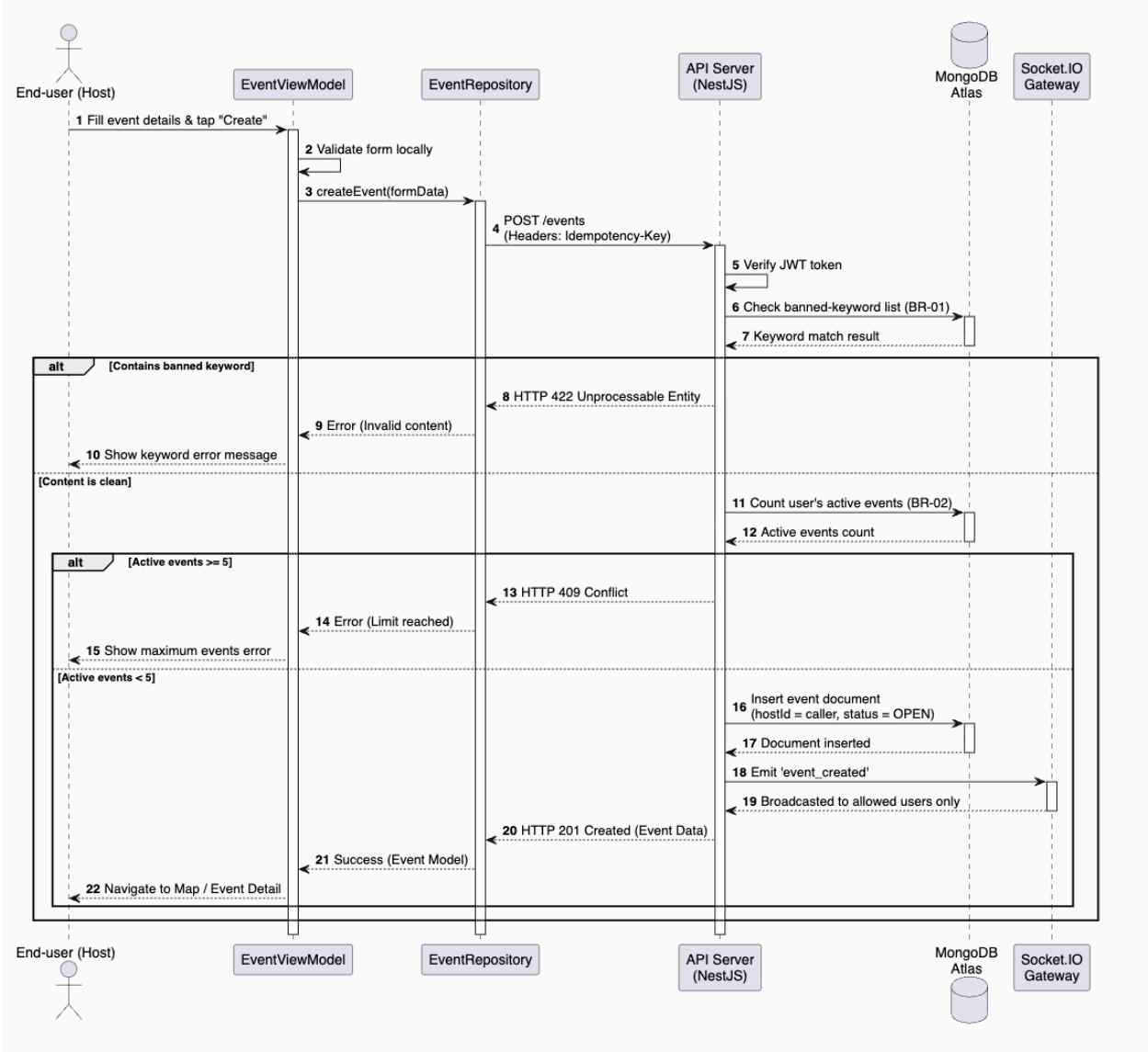
SEQ-02: Toggle Ghost Mode (UC-07).

1. The user taps the toggle; the app stops location updates and shows the indicator immediately.

2. PUT /me/privacy saves the flag and deletes the user's location document.

3. The server emits a hide event and friends' maps remove the marker (within 5 s).

4. Without network, the change stays pending and is retried; no location is sent meanwhile.



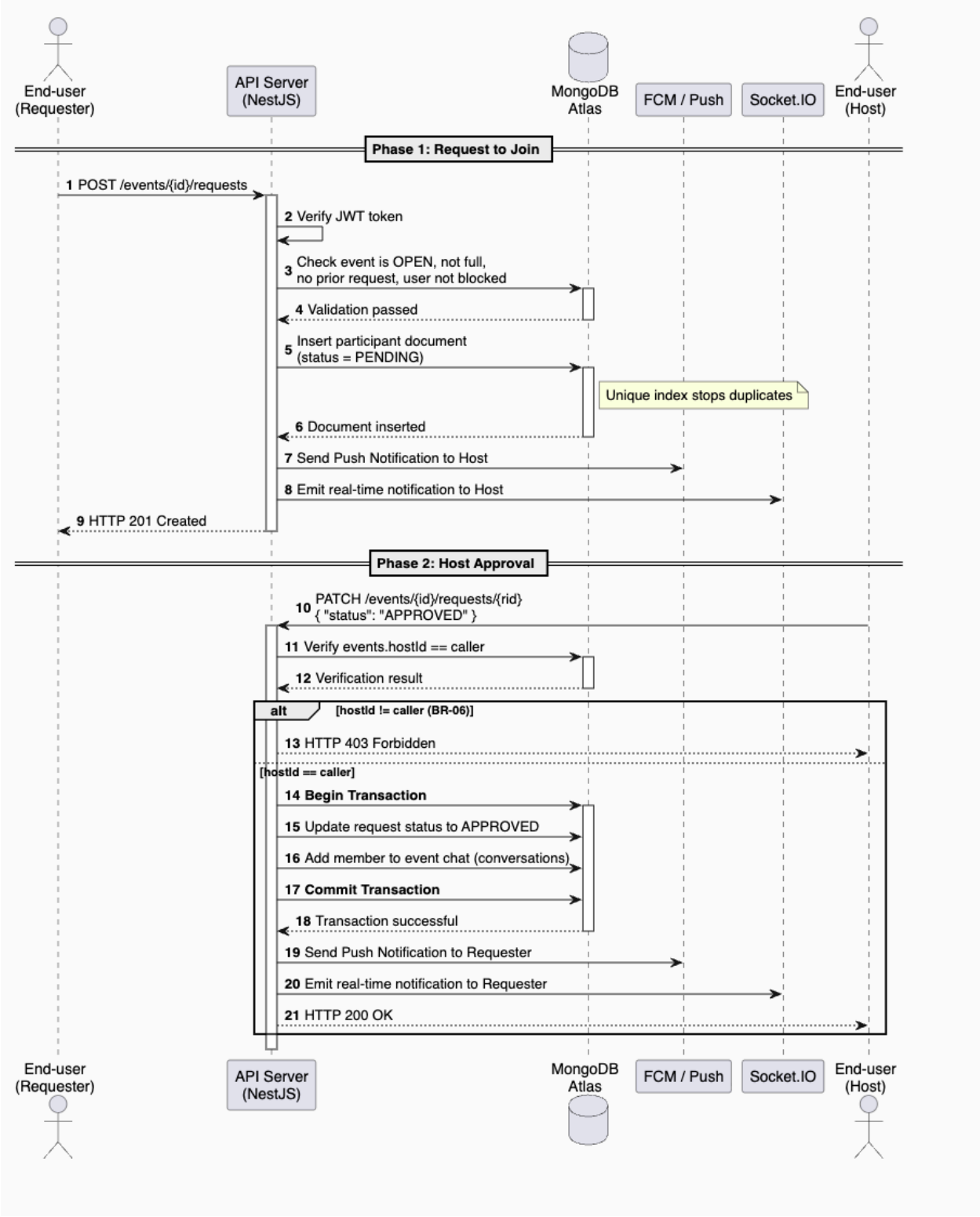
SEQ-03: Create event (UC-11).

1. The ViewModel validates the form and calls POST /events with an idempotency key.

2. The server validates the input and checks the banned-keyword list; a match returns 422 and nothing is saved (BR-01).

3. It counts the user's active events; a sixth returns 409 (BR-02).

4. Otherwise it inserts the event with host_id equal to the caller and status OPEN, and emits an event only to allowed users.



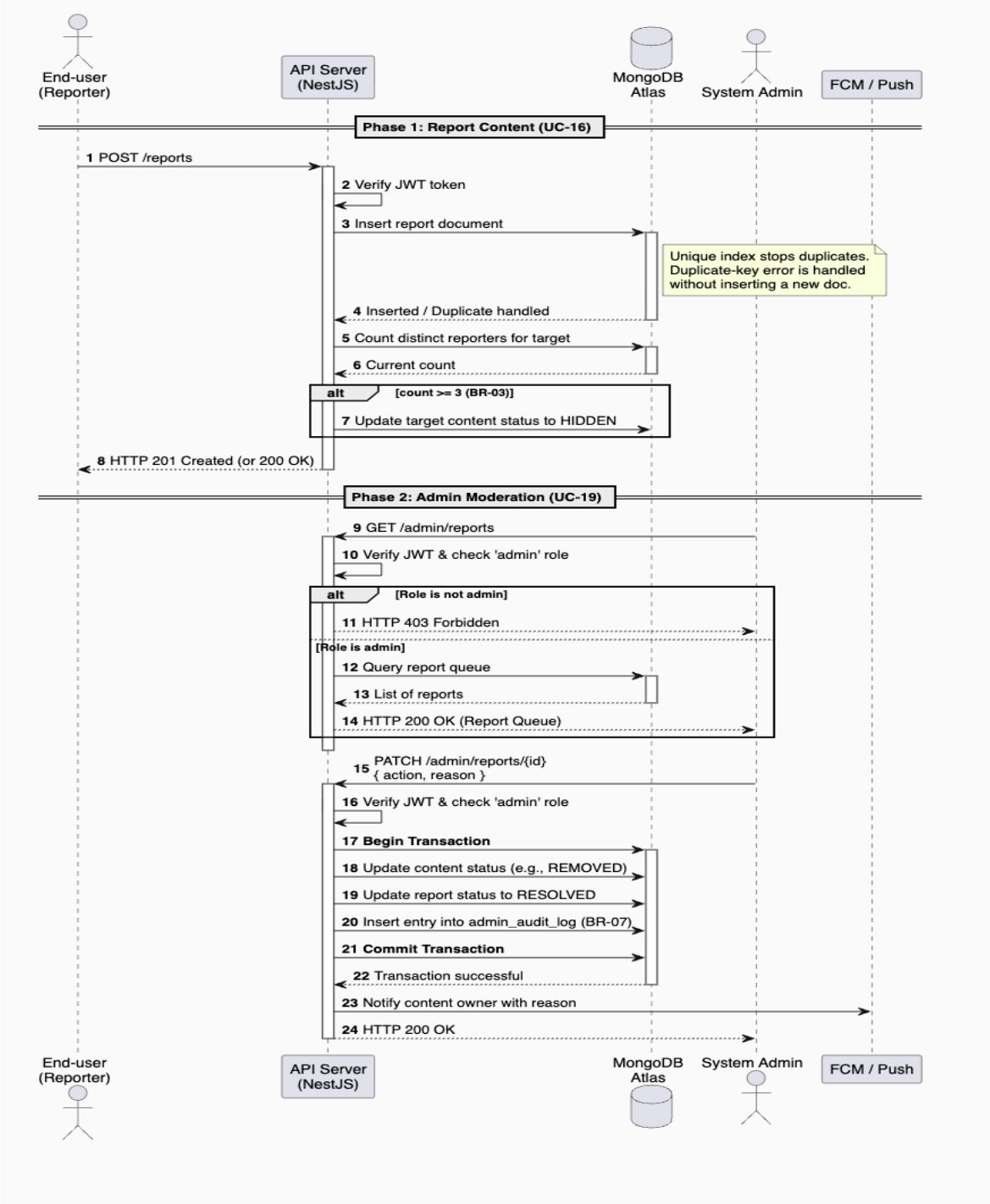
SEQ-04: Request to join and host approval (UC-09, UC-11). *[Insert Figure 7]*

1. POST /events/{id}/requests checks that the event is OPEN, not full, has no earlier request, and that the host has not blocked the user.

2. A PENDING participant document is inserted (a unique index stops duplicates) and the host is notified by push and socket.

3. The host approves with PATCH /events/{id}/requests/{rid}; the server verifies that host_id equals the caller, otherwise it returns 403 (BR-06).

4. In one transaction the request becomes APPROVED and the member is added to the event chat; if the maximum is reached the event becomes FULL, and the member is notified.



SEQ-05: Report and moderate content (UC-16, UC-19).

1. POST /reports inserts one report per user and target; a duplicate-key error is acknowledged without a new document.

2. The server counts distinct reporters; at 3 the content becomes HIDDEN (BR-03).

3. The Admin dashboard calls GET /admin/reports; the server checks the admin role and returns 403 otherwise.

4. The Admin's action PATCH /admin/reports/{id} runs in one transaction: it updates the content and the report and inserts the audit entry (BR-07); the owner is then notified with the reason.

4.7 How the architecture meets the non-functional requirements

NFR	Architectural decision
NFR-01, NFR-21	One combined endpoint for the map; 2dsphere indexes; cursor-based pagination of 20 posts on an index of status and created_at; show cached data first, then refresh
NFR-04, NFR-09	Server-side visibility filter before every location read and every socket emit; Ghost Mode deletes the document and emits a hide event
NFR-05, NFR-06	Repository merges API data with a local cache; idempotency keys; offline queue for chat
NFR-07, NFR-08	Hashed passwords in a separate collection; short-lived access tokens; hashed, rotated refresh tokens; login rate limit
NFR-14	One latest document per user and a TTL index; no history collection
NFR-15, NFR-16	Authentication middleware and ownership or role checks on every route; database connection only on the server; insert-only audit collection
NFR-18	MVVM with one-way dependencies; ViewModels and Repositories unit-tested; authorisation rules covered by automated tests
NFR-20	Date of birth stored at registration; the API refuses to enable location sharing for an account under 16 until guardian consent is recorded (exact legal rule to be verified)

4.8 Deployment and architectural risks

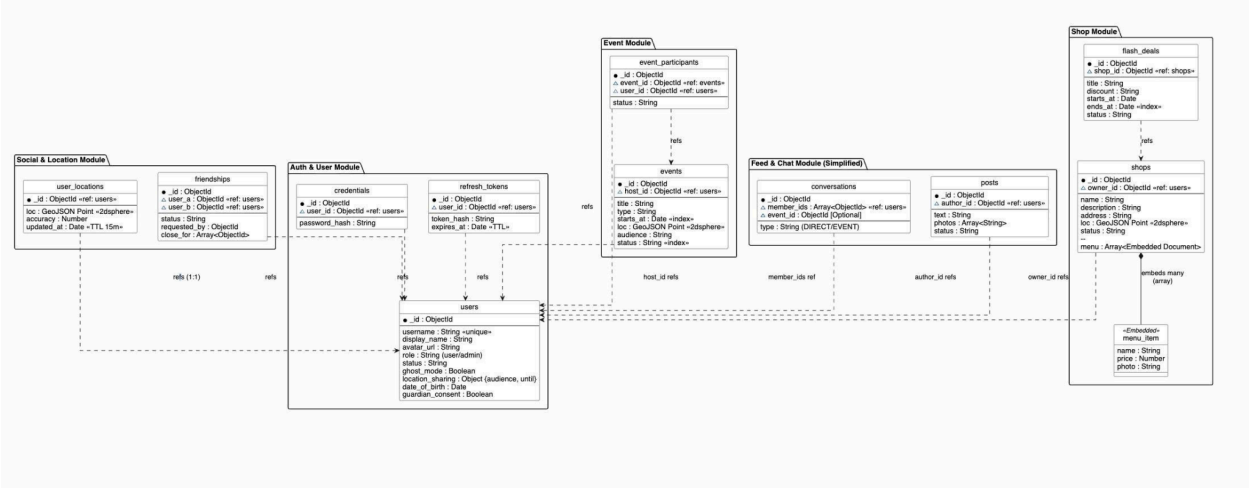
Deployment. The API server runs as a container on a cloud platform, with MongoDB Atlas as a managed cluster; development and production are separate environments with separate secrets. Index and validation definitions are kept in versioned scripts. CI runs lint and tests on every push. Crash reporting provides the figure for the stability target.

Risk	Mitigation
Authorisation exists only in code: one missing check leaks data	Put all checks in one authorisation layer; write tests with unauthorised accounts for every route; code review
No foreign keys: orphaned or inconsistent documents	Transactions for multi-document changes; unique indexes; schema validation; cleanup script on account deletion
The team must build authentication, real-time delivery and file handling	Use established libraries; limit scope to username and Google sign-in; review security-critical code

Risk	Mitigation
Single API instance holds all sockets	Enough for about 50 concurrent users; add a shared adapter before running several instances [verify]
TTL cleanup is not instant	State the 15-minute expiry as approximate; the API also ignores locations older than 15 minutes when reading
Wrong GeoJSON coordinate order or missing index gives wrong or slow geo results	Use longitude then latitude; add 2dsphere indexes; test radius queries with known points
Atlas tier or hosting plan limits	Check connection and storage limits for the chosen plans before committing to NFR-11 and NFR-12
Map quota and cost	Restrict the API key per app; cluster markers when dense; consider OpenStreetMap
Store review of background location	Ask for background location only when the user opts in and explain the purpose clearly
Battery drain from frequent location	Adaptive intervals; stop in Ghost Mode; background tracking can be turned off; measure against the 5% per hour target
Database credentials leak	Keep them only on the server in a secrets store; never in the app or dashboard

5. Data and API design

5.1 ERD



Core collections

Collection	Main fields	Indexes and notes
users	username, display_name, avatar_url, bio, role (user or admin), status (ACTIVE or SUSPENDED), ghost_mode, location_sharing (audience, until),	Unique username. Date of birth and guardian consent support NFR-20

Collection	Main fields	Indexes and notes
	notification_settings, date_of_birth, guardian_consent, google_id (Google accounts only)	
credentials	user_id, password_hash	Separate from users and never returned by any API; absent for Google-only accounts
refresh_tokens	user_id, token_hash, expires_at	TTL on expires_at; rotated on use
friendships	user_a, user_b, status (PENDING or ACCEPTED), requested_by, close_for	Unique pair. The BLOCKED relationship of UC-05 is stored in blocks, so two users can block each other
blocks	blocker_id, blocked_id	Unique pair
user_locations	_id = user_id, loc (GeoJSON point), accuracy, updated_at	2dsphere on loc; TTL 900 s on updated_at
events	host_id, title, type, description, starts_at, loc, audience (EVERYONE or CLOSE_FRIENDS), max_participants, status (OPEN, FULL, CANCELLED, ENDED, HIDDEN, REMOVED)	2dsphere on loc; index on status and starts_at
event_participants	event_id, user_id, status (PENDING, APPROVED, REJECTED, CANCELLED, LEFT, KICKED)	Unique on event_id and user_id
shops	owner_id, name, description, address, loc, status (PENDING, APPROVED, REJECTED), reviewed_by, reject_reason, menu (embedded items: name, price, photo)	2dsphere on loc. The small menu is embedded because it is always read with the shop
flash_deals	shop_id, title, discount, starts_at, ends_at, status (ACTIVE, ENDED, CANCELLED)	Index on shop_id and ends_at. A scheduled job sets ENDED, and the API also ignores deals whose ends_at has passed
posts, post_likes, post_comments, saved_posts	author_id, text (up to 500 characters), photos (up to 4 URLs), status (ACTIVE, HIDDEN, REMOVED), counters	Index on status and created_at for paginated feeds (20 per page). Unique on post_id and user_id for likes. Author name and avatar are looked up at read time, not copied
conversations, messages	conversation type (DIRECT or EVENT), member_ids, event_id; message sender_id, text, created_at	Index on conversation_id and created_at; only members may read
notifications, device_tokens	user_id, type, payload, read_at; token, platform	Index on user_id and created_at
reports	reporter_id, target_type, target_id, reason, note, status (OPEN or RESOLVED)	Unique on reporter_id, target_type and target_id
banned_keywords	keyword	Checked by BR-01
admin_audit_log	admin_id, action, target_id, reason, created_at	Insert-only: the API never updates or deletes it. A database user with insert and read rights only on this collection can enforce this [verify]

The database has no foreign keys, so the API enforces references. Where several documents must change together (approve a request and add the member to the chat, remove content and write the audit entry, delete an account), the server uses MongoDB multi-document transactions, which need the replica set that Atlas provides. Collections also carry JSON Schema validation to reject malformed documents.

5.2 Local Storage Strategy

To satisfy the offline requirements (NFR-05, NFR-06) and security constraints (NFR-07, NFR-08) within the Flutter application, the local storage strategy is divided into four distinct tiers based on data sensitivity and structure. The Repository layer abstracts these storage mechanisms from the ViewModels, acting as the single source of truth.

Tier 1: Secure Storage (Encrypted Key-Value)
Target Data: JWT Access Tokens, Refresh Tokens, Google ID Tokens
Mechanism: OS-backed encrypted storage (Keychain for iOS, Keystore for Android) **Flutter Package:** flutter_secure_storage
Justification: Satisfies NFR-07 and NFR-08 as tokens are highly sensitive credentials and must never be stored in plain text

Tier 2: App Preferences (Plain Key-Value)
Target Data: Notification toggles, Ghost Mode status, theme preferences, and first launch flags
Mechanism: Standard key-value persistence
Flutter Package: shared_preferences
Justification: Fast read and write performance for lightweight non-sensitive UI settings and configuration flags

Tier 3: Structured Data and Offline Cache (Local Database)
Target Data: Cached friends locations, nearby events, recent feed posts, offline chat messages and pending write actions (NFR-05, NFR-06)
Mechanism: Fast local NoSQL database
Flutter Package: Hive or Isar
Justification: Since the backend uses MongoDB using a local NoSQL database in Flutter allows for a seamless mapping of JSON documents to serve the Repository layer when the network is unavailable

Tier 4: Media Cache (File System)
Target Data: User avatars, shop menu images, and feed photos
Mechanism: Temporary file system caching with automatic eviction policies
Flutter Package: cached_network_image or flutter_cache_manager
Justification: Significantly reduces mobile data usage (section 3.2) and ensures images load instantly from the disk when users scroll through the feed or interact with the map

5.3 API Design

5.3.1. Conventions

Base URL: https://api.zenzy.app/v1

Authentication: Authorization: Bearer <access_token>. Access token lives 15 min; refresh token is rotated and lives 30 days (NFR-08). Public routes: register, login, Google sign-in, refresh.

Roles: `USER` and `ADMIN`. Routes under `/admin` return 403 for non-admins. Owner and host checks (BR-06) are done on the server and return 403.

Errors: every error returns `code`, `message` and `timestamp`. Status codes: 400 invalid input, 401 not authenticated, 403 not allowed, 404 not found (also used when a hidden or blocked item must not be revealed), 409 conflict, 422 banned keyword (BR-01), 429 rate limit.

Privacy: every location read and every socket event is filtered on the server (NFR-09).

Build order for the course project: implement the *Must* use cases first (auth, profile, privacy and location, map, friends, chat, events, reports and moderation), then the *Should* ones (shops, flash deals, feed, search).

5.3.2. Authentication and account (UC-01, UC-02, UC-17)

Method and path	Description	Request (main fields)	Response and errors	UC
POST `/auth/register`	Create account (public)	`username`, `password` (min 8), `date_of_birth`	201 user and tokens. 400 invalid, 409 `USERNAME_TAKEN`	UC-01
POST `/auth/login`	Log in (public). Also used by the admin dashboard	`username`, `password`	200 user (with `role`) and tokens. 401 wrong credentials, 403 suspended, 429 too many attempts	UC-01, UC-17

POST `/auth/google`	Google sign-in (public)	`id_token`; on first sign-in also `username`, `date_of_birth`	200 user and tokens. 400 `PROFILE_REQUIRED` (new account without username), 401 invalid token	UC-01
POST `/auth/refresh`	Rotate tokens (public)	`refresh_token`	200 new token pair. 401 invalid or expired	UC-01
POST `/auth/logout`	End the session	`refresh_token`	204	UC-01, UC-17
GET `/me`	Own profile and settings	none	200 profile, privacy, notification settings	UC-02
PATCH `/me`	Edit profile	`display_name`, `avatar_url`, `bio`	200 profile. 400 invalid	UC-02
PUT `/me/password`	Change password (hidden for Google-only accounts)	`current_password`, `new_password`	204. 400 `WRONG_PASSWORD`, 409 `NO_PASSWORD`	UC-02
DELETE `/me`	Delete account; data and media removed within 72 h	`password` or Google `id_token`	202	NFR-10
PUT `/me/guardian-consent`	Record parental consent for users under 16	`confirmed`	200	NFR-20

GET `/users/{user_id}`	View another user's profile	none	200 public profile and `friendship_status`. 404 if blocked	UC-02, UC-05
GET `/users`	Search users by username	`username`	200 list (blocked users excluded)	UC-05

5.3.3. Privacy, location and map (UC-03, UC-04, UC-07, UC-23)

Method and path	Description	Request (main fields)	Response and errors	UC
GET `/me/privacy`	Read sharing settings	none	200 settings	UC-23
PUT `/me/privacy`	Change sharing settings. Turning Ghost Mode on deletes the stored location and emits `location:hide`	`ghost_mode`, `audience` (`FRIENDS` or `CLOSE_FRIENDS`), `share_until` (null or time; presets 1 h, 8 h, 24 h), `precise`, `hidden_from` (user IDs)	200 settings, effective within 5 s. 403 `GUARDIAN_CONSENT_REQUIRED` (under 16)	UC-07, UC-23
PUT `/me/location`	Send current position (every 15 s moving, 2 min still). Overwrites the previous one;	`latitude`, `longitude`, `accuracy`	204. 409 `SHARING_OFF` if Ghost Mode or sharing is off	UC-03, BR-05

	expires after 15 min			
GET `/map/nearby`	Friends, events and shops in one call	`latitude`, `longitude`, `radius` (m, default 5000), `types` (`friends,events,shops`)	200 `friends` (position and `updated_at`), `events`, `shops` (with deal badge). Only data the caller may see	UC-03, UC-08, UC-13
GET `/search`	Search shops and events by keyword	`q`, `type` (`SHOP` or `EVENT`), `latitude`, `longitude`, cursor	200 list sorted by distance; empty list if no match	UC-04

5.3.4 Friends and blocking (UC-05)

Method and path	Description	Request (main fields)	Response and errors	UC
GET `/friends`	List friends or pending requests	`status` (`ACCEPTED`, `PENDING`), `q`	200 list with `close_friend` flag	UC-05
POST `/friends/requests`	Send friend request	`receiver_id`	201 PENDING. 409 duplicate, 403 blocked	UC-05
PATCH `/friends/requests/{request_id}`	Recipient accepts or rejects	`action` (`ACCEPT`, `REJECT`)	200. 403 not the recipient	UC-05

DELETE `/friends/{user_id}`	Unfriend	none	204	UC-05
PUT `/friends/{user_id}/close`	Add or remove a close friend	`close` (boolean)	200	UC-05
GET `/blocks`	List blocked users	none	200	UC-05
POST `/blocks`	Block a user; the friendship ends	`user_id`	201	UC-05
DELETE `/blocks/{user_id}`	Unblock; the friendship is not restored	none	204	UC-05

5.3.5 Events (UC-08 to UC-11)

Method and path	Description	Request (main fields)	Response and errors	UC
GET `/events`	List events near a point	`latitude`, `longitude`, `radius`, `type`, `when` (`NOW`, `SCHEDULED`)	200 list. Ended events and Close Friends events the caller may not see are excluded	UC-08
GET `/events/{event_id}`	Event detail	none	200 with `host`, `participants_count`, `spots_left`,	UC-08

			`my_status`. 404 if not visible	
POST `/events`	Create event or live pin (`Idempotency-Key`)	`title` (3 to 80), `type` (`EVENT`, `SPORTS`, `STUDY`, `OTHER`), `description`, `latitude`, `longitude`, `place_name`, `start_time`, `end_time`, `max_participants`, `audience` (`EVERYONE`, `CLOSE_FRIENDS`)	201 event (OPEN). 400 invalid, 409 `EVENT_LIMIT` (sixth active event, BR-02), 422 banned keyword (BR-01)	UC-11
PATCH `/events/{event_id}`	Edit event (host only)	any create field	200. 403 not host	UC-11
POST `/events/{event_id}/cancel`	Cancel event; members are notified	`reason` (optional)	200 CANCELLED. 403 not host	UC-11
GET `/events/{event_id}/participants`	Host sees all statuses; others see only APPROVED	`status`	200 list	UC-08, UC-11

POST `/events/{event_id}/requests`	Request to join (`Idempotency-Key`)	none	201 PENDING; host notified within 10 s. 409 `EVENT_FULL`, `DUPLICATE_REQUEST`, `EVENT_ENDED`. 403 blocked by host	UC-09
PATCH `/events/{event_id}/requests/{request_id}`	Host approves or rejects. One transaction: update status, add member to the event chat, set FULL when the maximum is reached	`status` (`APPROVED`, `REJECTED`)	200. 403 not host (BR-06)	UC-11
DELETE `/events/{event_id}/requests/me`	Cancel own request or leave the event; seat is released and FULL becomes OPEN	none	204	UC-10
DELETE `/events/{event_id}/members/{user_id}`	Host removes a member (KICKED)	none	204. 403 not host	UC-11

5.3.6 Shops and flash deals (UC-12 to UC-15)

Method and path	Description	Request (main fields)	Response and errors	UC
GET `/shops/{shop_id}`	Shop profile, menu and active deal	none	200. 404 unless APPROVED (or the caller is the owner)	UC-12, UC-13
POST `/shops`	Create shop; stays PENDING until an admin approves it (BR-04)	`name`, `description`, `address`, `latitude`, `longitude`, `menu` (list of `name`, `price`, `photo_url`)	201 PENDING. 422 banned keyword	UC-14
PATCH `/shops/{shop_id}`	Edit shop and menu (owner). A REJECTED shop is resubmitted as PENDING	any create field	200. 403 not owner	UC-14
GET `/me/shops`	Own shops with status and reject reason	none	200 list	UC-14
GET `/shops/{shop_id}/deals`	Flash deals of a shop	`status` (default `ACTIVE`)	200 list. 404	UC-13

POST `/shops/{shop_id}/deals`	Create flash deal (shop must be APPROVED)	`title`, `discount`, `start_time`, `end_time`	201 ACTIVE. 400 end not after start. 403 shop not approved	UC-15
PATCH `/shops/{shop_id}/deals/{deal_id}`	Edit a deal or end it early	fields, or `status` = `CANCELLED`	200. 403 not owner	UC-15

5.3.7 Feed and uploads (UC-21)

Method and path	Description	Request (main fields)	Response and errors	UC
GET `/posts`	Feed, 20 per page, newest first	`feed` (`FOR_YOU`, `FOLLOWING`, `NEARBY`), `latitude` and `longitude` (for `NEARBY`), cursor	200. Hidden posts and posts of users who blocked the caller are excluded	UC-21
POST `/posts`	Create post (`Idempotency-Key`)	`text` (max 500), `photo_urls` (max 4), `place_name`	201. 422 banned keyword (BR-01)	UC-21
PATCH `/posts/{post_id}`	Edit own post	`text`	200. 403	UC-21
DELETE `/posts/{post_id}`	Delete own post	none	204. 403	UC-21

PUT and DELETE `/posts/{post_id}/like`	Like or unlike	none	204	UC-21
GET and POST `/posts/{post_id}/comments`	List or add comments	`text` (POST)	200 / 201	UC-21
PUT and DELETE `/posts/{post_id}/save`	Save or unsave	none	204	UC-21
GET `/me/saved-posts`	Saved posts	cursor	200	UC-21
POST `/uploads/sign`	Get a signed upload URL for an avatar, post photo or shop image (max 500 KB)	`purpose`, `content_type`	200 `upload_url` and `public_url`	UC-02, UC-14, UC-21

5.3.8 Chat (UC-06)

Method and path	Description	Request (main fields)	Response and errors	UC
GET `/conversations`	Conversation list	`type` (`DIRECT`, `GROUP`, `EVENT`)	200 with `last_message`, `unread_count`, `muted`	UC-06

POST `/conversations`	Start a direct chat with a friend or create a group. Event chats are created automatically with the event	`type`, `member_ids`, `name` (group)	201. 403 not friends or blocked	UC-06
GET `/conversations/{id}/messages`	Message history, newest first	`limit`, `before`	200. 403 not a member	UC-06
POST `/conversations/{id}/messages`	Send a message (`Idempotency-Key`); delivered by socket within 2 s	`content`	201. 403 not a member or blocked	UC-06
POST `/conversations/{id}/read`	Mark as read	none	204	UC-06
PATCH `/conversations/{id}`	Mute or unmute	`muted`	200	UC-06
DELETE `/conversations/{id}`	Remove from own list	none	204	UC-06

5.3.9 Notifications (UC-22)

Method and path	Description	Request (main fields)	Response and errors	UC

GET `/notifications`	List notifications	cursor	200 with `type`, `payload`, `read_at`	UC-22
POST `/notifications/read-all`	Mark all as read	none	204	UC-22
PATCH `/notifications/{id}`	Mark one as read	`read`	200	UC-22
GET and PUT `/me/notification-settings`	Turn push on or off per type	map of type to boolean	200	UC-22
PUT and DELETE `/me/device-tokens`	Register or remove the push token (remove at logout)	`token`, `platform` (`ANDROID`, `IOS`)	204	UC-22

5.3.10 Reports and administration (UC-16 to UC-20)

Method and path	Description	Request (main fields)	Response and errors	UC

POST `/reports`	Report content or a user (`Idempotency-Key`)	`target_type` (`POST`, `EVENT`, `SHOP`, `USER`), `target_id`, `reason` (`SPAM`, `SCAM`, `HARASSMENT`, `ILLEGAL`, `SENSITIVE`, `OTHER`), `note`	201. 200 if already reported by the caller (no duplicate). 400 own content. The third distinct reporter hides the content (BR-03)	UC-16
GET `/admin/reports`	Report queue sorted by count and time (admin)	`status`, cursor	200. 403 non-admin	UC-19
PATCH `/admin/reports/{report_id}`	Review a report (admin). One transaction: update content, close report, write audit log (BR-07), notify the owner	`action` (`DISMISS`, `REMOVE`, `REMOVE_WARN`, `REMOVE_SUSPEND`), `reason`	200. 403	UC-19, UC-20
GET `/admin/shops`	Shops waiting for review (admin)	`status` (default `PENDING`)	200 list	UC-18
PATCH `/admin/shops/{shop_id}`	Approve or reject a shop	`decision` (`APPROVE`, `REJECT`), `reason` (required for reject)	200. Owner notified	UC-18
GET `/admin/users`	Search accounts (admin)	`q`	200 with report counts	UC-20

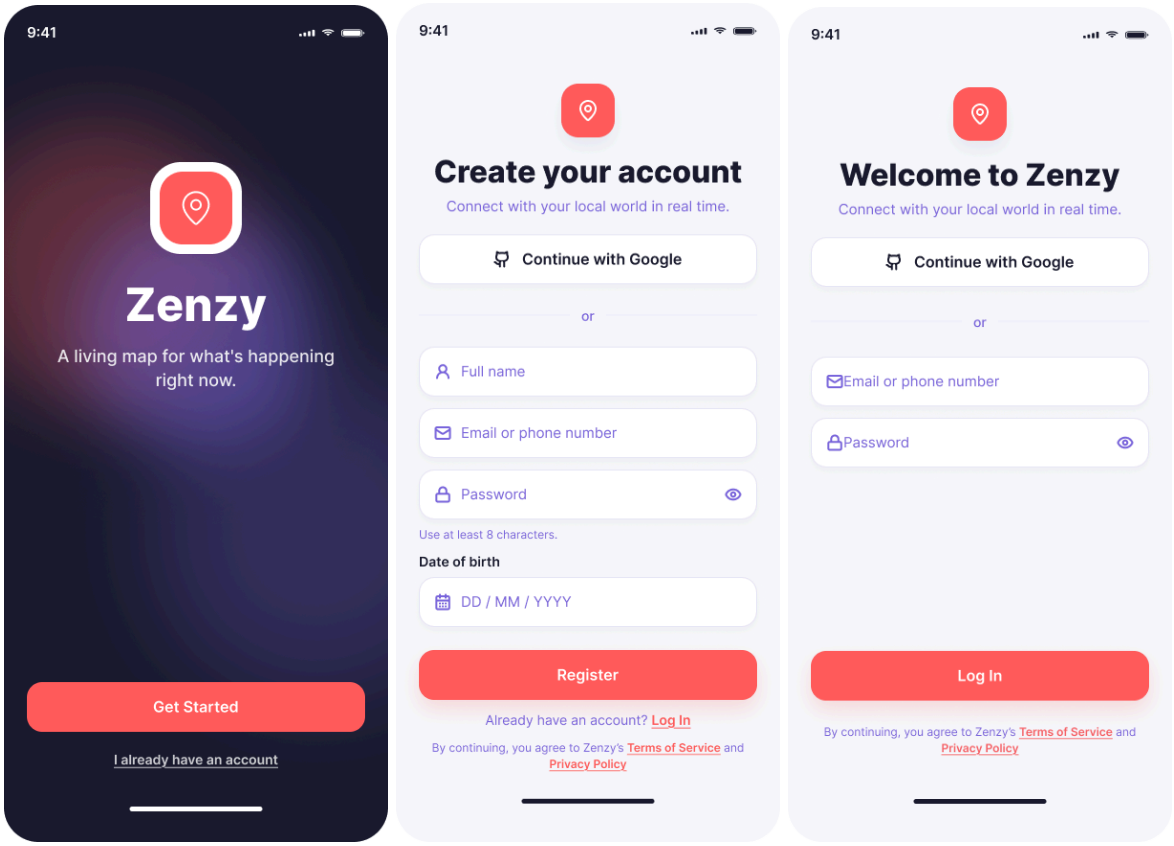
PATCH `/admin/users/{user_id}/status`	Suspend or restore an account	`status` (`SUSPENDED`, `ACTIVE`), `reason`	200. Audit logged, user notified	UC-20

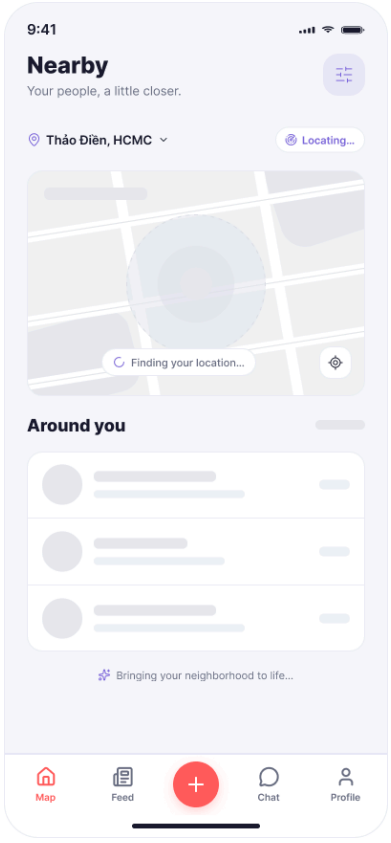
5.3.11 Real-time events (Socket.IO)

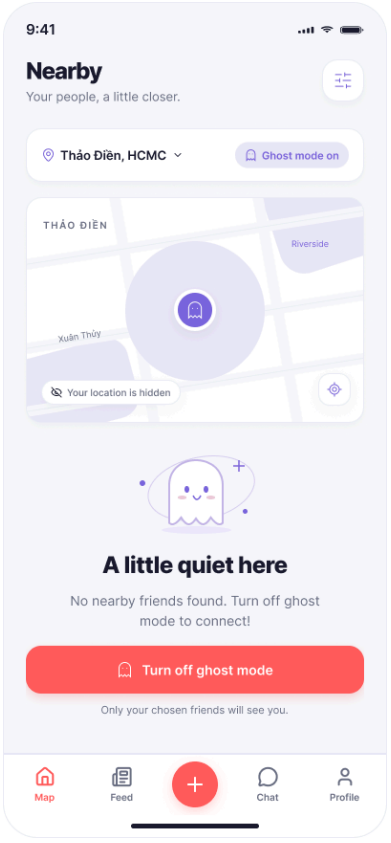
The app connects with `auth: { token: <access_token> }`. The server puts the socket in the room `user:<id>` and in the rooms of the user's conversations. If the socket drops, the app falls back to REST at the same interval.

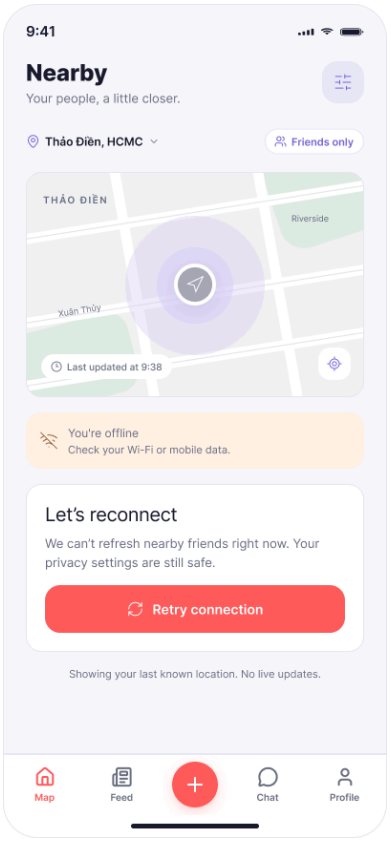
Event	Direction	Payload	UC
`location:update`	server to client	`user_id`, `latitude`, `longitude`, `updated_at` (only to allowed friends)	UC-03
`location:hide`	server to client	`user_id` (Ghost Mode on, sharing ended, or blocked)	UC-07
`event:created`, `event:updated`	server to client	event summary (only to allowed users)	UC-08
`chat:message`	server to client	`conversation_id` and the message	UC-06
`notification:new`	server to client	the notification	UC-22

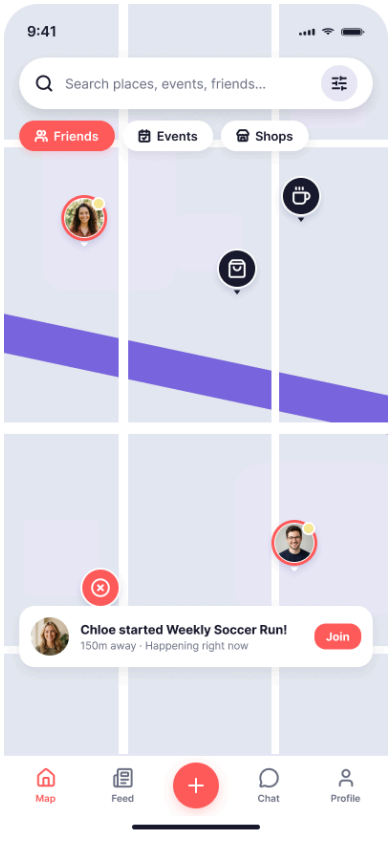
6. UI/UX design and mockups

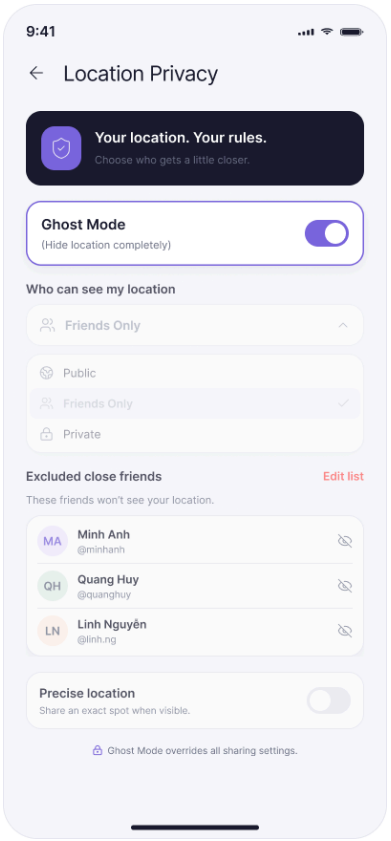


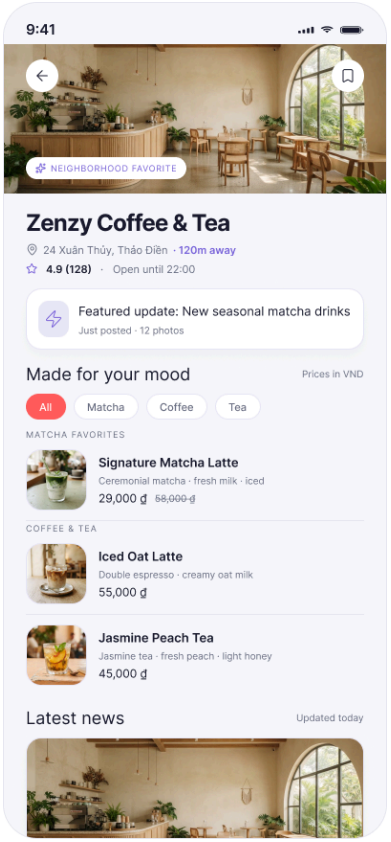

9:41
Nearby
Your people, a little closer.
Thảo Điền, HCMC
Locating...
Finding your location...
Around you
Bringing your neighborhood to life...
Map
Feed
Chat
Profile


9:41
Nearby
Your people, a little closer.
Thảo Điền, HCMC
Ghost mode on
THẢO ĐIỀN
Riverside
Xuân Thủy
Your location is hidden
A little quiet here
No nearby friends found. Turn off ghost mode to connect!
Turn off ghost mode
Only your chosen friends will see you.
Map
Feed
Chat
Profile


9:41
Nearby
Your people, a little closer.
Thảo Điền, HCMC
Friends only
THẢO ĐIỀN
Riverside
Xuân Thủy
Last updated at 9:38
You're offline
Check your Wi-Fi or mobile data.
Let's reconnect
We can't refresh nearby friends right now. Your privacy settings are still safe.
Retry connection
Showing your last known location. No live updates.
Map
Feed
Chat
Profile


9:41
Search places, events, friends...
Friends
Events
Shops
Chloe started Weekly Soccer Run!
150m away · Happening right now
Join
Map
Feed
Chat
Profile


9:41
Location Privacy
Your location. Your rules.
Choose who gets a little closer.
Ghost Mode
(Hide location completely)
Who can see my location
Friends Only
Public
Friends Only
Private
Excluded close friends
Edit list
These friends won't see your location.
MA
Minh Anh
@minhanh
QH
Quang Huy
@quanghuy
LN
Linh Nguyên
@linh.ng
Precise location
Share an exact spot when visible.
Ghost Mode overrides all sharing settings.


9:41
NEIGHBORHOOD FAVORITE
Zenzy Coffee & Tea
24 Xuân Thủy, Thảo Điền · 120m away
4.9 (128) · Open until 22:00
Featured update: New seasonal matcha drinks
Just posted · 12 photos
Made for your mood
Prices in VND
All
Matcha
Coffee
Tea
MATCHA FAVORITES
Signature Matcha Latte
Ceremonial matcha · fresh milk · iced
29,000 ₫ 58,000 ₫
COFFEE & TEA
Iced Oat Latte
Double espresso · creamy oat milk
55,000 ₫
Jasmine Peach Tea
Jasmine tea · fresh peach · light honey
45,000 ₫
Latest news
Updated today

9:41



Weekly Soccer Run at Riverside Courts

A friendly 5-a-side with the neighborhood crew. Bring your boots, water & good energy.

Saturday, 7 November
17:00–18:30 · GMT+7 · Every Saturday
Free



Riverside Courts · Court 2
206 Nguyễn Văn Hưởng, Thảo Điền
Directions

DK Organized by **Chloe** Message

Participants · 8 View all

MA QH LN +5 **8 going · 4 spots left**
2 friends are joining

- MA **Minh Anh** @minhanh · Your friend ✓
- QH **Quang Huy** @quanghuy · Your friend ✓
- LN **Linh Nguyên** @linh.ng · Joined today ✓

Join Event / RSVP

9:41

zenzy

For You | Following | Nearby

Liam Chen
12 mins ago • Soho District

Checked out the new artisanal match store today. Incredible vibes and local community atmosphere! ☕✨



142 28

Sloane Miller
1 hr ago • Riverside Park

Weekly community run was a major success! Over 40 people showed up today 🏃‍♂️☀️



Map | Feed | + | Chat | Profile

9:41

Zoe Anderson
Online now

Today

Hey Zoe! Fancy a walk in the park later? 9:20 AM

Yes please! Riverside Park sounds lovely 🌿 9:21 AM

Riverside Park
Thảo Điền · Main entrance
View on map 9:22 AM

Love that spot. Meet at the main entrance? 9:23 AM

Are we still meeting at the park at 5? 9:39 AM

Yes! See you at 5 😊 9:41 AM · Seen

Message Zoe...

9:41

Notifications

Mark all read

TODAY

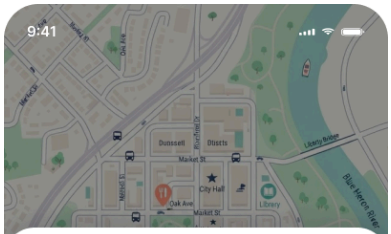
- Zoe Anderson sent you a **new message** · 2 mins ago
- Maya RSVP'd to your upcoming event **Clay Workshop** · 2 hrs ago
- Your friend Liam Chen is **nearby in Soho** · 4 hrs ago

THIS WEEK

- Local Deal: 20% off at **Blue Bottle Coffee** · 2 days ago
- Marcus Vance and 3 others **liked your post** · 4 days ago

Map | Feed | + | Chat | Profile

9:41



Create Live Pin

Event | Sports | Study | Deal

Pin Title
Need +2 for basketball

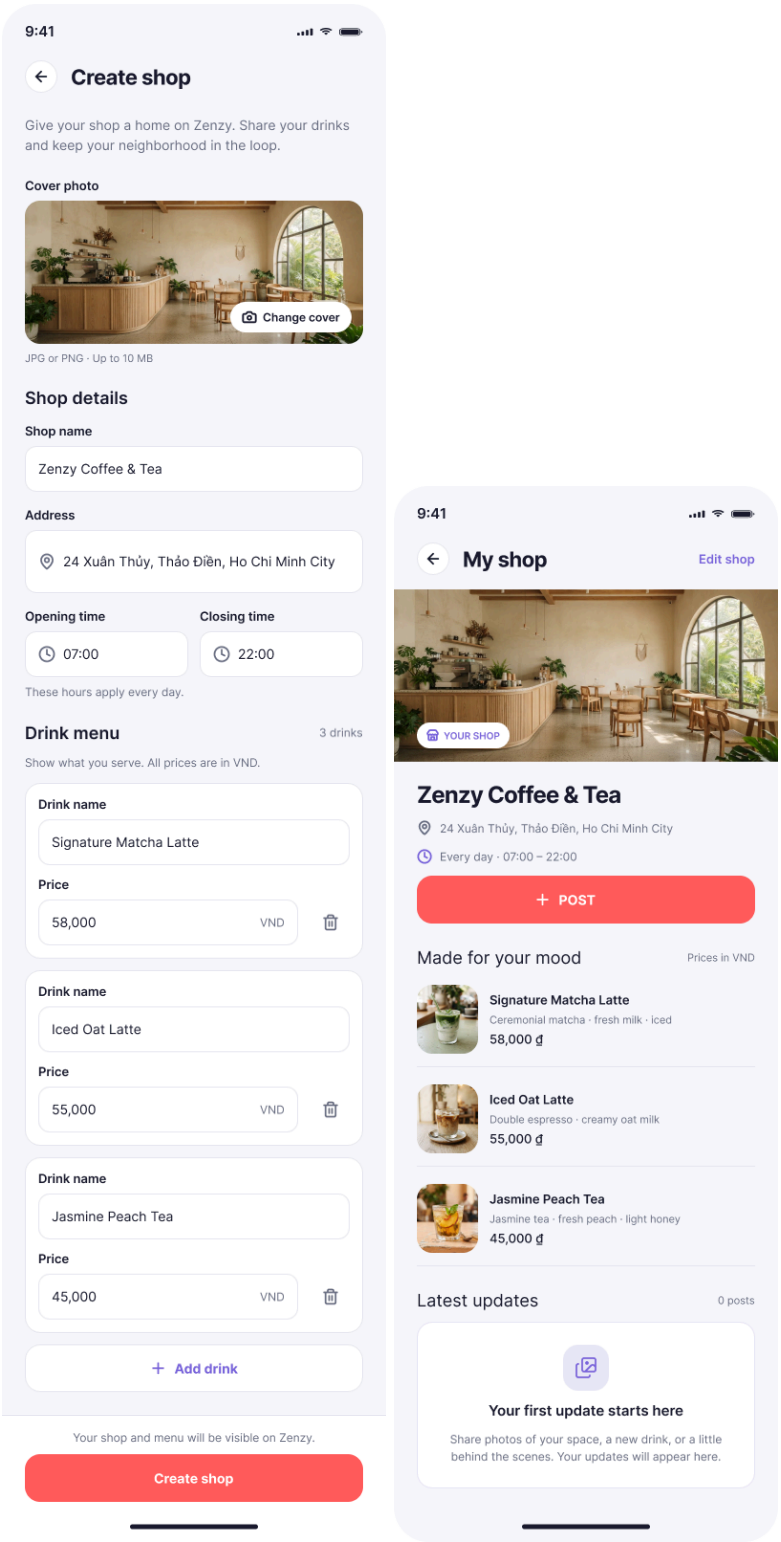
Time
5:30 PM (Today)

Location
Riverside Courts

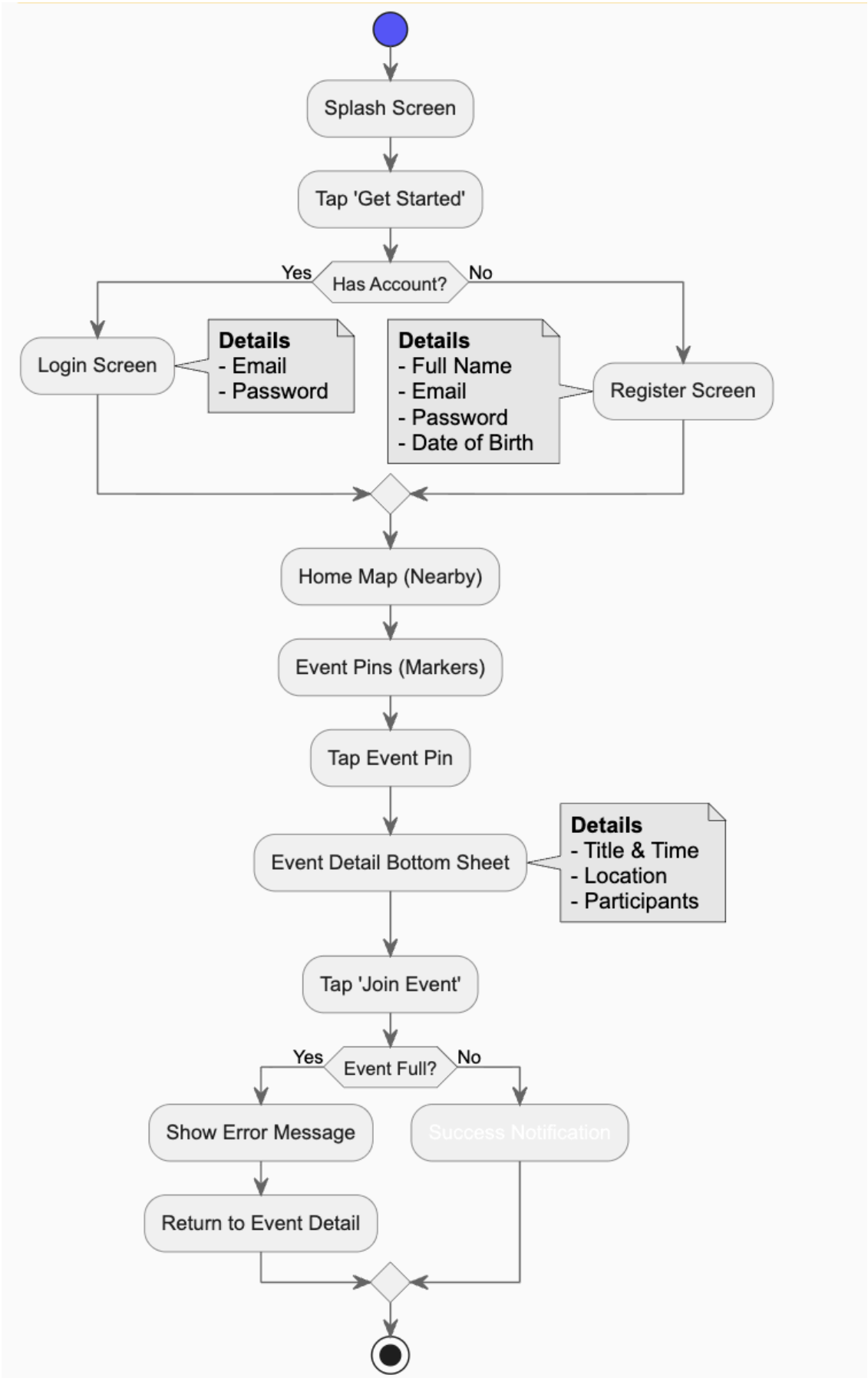
Description
Casual pick-up game, friendly vibes. Bring a black/white shirt if possible!

Who can see this?
Everyone nearby | Close Friends

Post to Map


9:41
Create shop
Give your shop a home on Zenzy. Share your drinks and keep your neighborhood in the loop.
Cover photo
Change cover
JPG or PNG · Up to 10 MB
Shop details
Shop name
Zenzy Coffee & Tea
Address
24 Xuân Thủy, Thảo Điền, Ho Chi Minh City
Opening time
07:00
Closing time
22:00
These hours apply every day.
Drink menu
3 drinks
Show what you serve. All prices are in VND.
Drink name
Signature Matcha Latte
Price
58,000
VND
Drink name
Iced Oat Latte
Price
55,000
VND
Drink name
Jasmine Peach Tea
Price
45,000
VND
Add drink
Your shop and menu will be visible on Zenzy.
Create shop
9:41
My shop
Edit shop
YOUR SHOP
Zenzy Coffee & Tea
24 Xuân Thủy, Thảo Điền, Ho Chi Minh City
Every day · 07:00 – 22:00
POST
Made for your mood
Prices in VND
Signature Matcha Latte
Ceremonial matcha · fresh milk · iced
58,000 ₫
Iced Oat Latte
Double espresso · creamy oat milk
55,000 ₫
Jasmine Peach Tea
Jasmine tea · fresh peach · light honey
45,000 ₫
Latest updates
0 posts
Your first update starts here
Share photos of your space, a new drink, or a little behind the scenes. Your updates will appear here.

User flow


Splash Screen
Tap 'Get Started'
Has Account?
Yes
No
Login Screen
Details
- Email
- Password
Details
- Full Name
- Email
- Password
- Date of Birth
Register Screen
Home Map (Nearby)
Event Pins (Markers)
Tap Event Pin
Event Detail Bottom Sheet
Details
- Title & Time
- Location
- Participants
Tap 'Join Event'
Event Full?
Yes
No
Show Error Message
Success Notification
Return to Event Detail